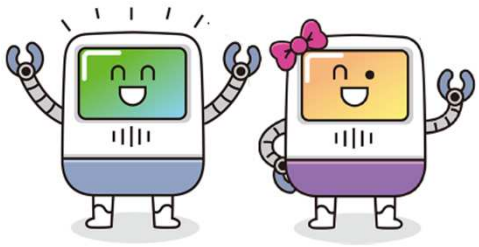
딥러닝

7장 텐서플로로 구현하는 딥러닝



07

텐서플로로 구현하는 딥러닝

학습목차

01 딥러닝을 위한 도구: 텐서플로
02 딥러닝 모델을 만들고 학습시키기
03 신경망 학습의 출력값과 정답의 차이를 알아보자

학습목표

- 딥러닝 모델을 만들고 이 모델을 학습시키는 방법과 그 단계별 내용을 이해한다.
- 필요에 따라 여러 개의 은닉층을 가진 딥러닝 모델을 생성할 수 있다.
- 딥러닝 모델을 학습시키기 위한 하이퍼파라미터의 의미를 이해한다.
- 소프트맥스 함수, 원-핫 인코딩을 이해하고 이를 딥러닝 모델에서 사용할 수 있다.



- 딥러닝을 위한 도구: 텐서플로
  - 텐서플로와 MNIST 데이터
  - 컴퓨터는 이미지를 어떻게 저장할까
  - MNIST 데이터 시각화하기
- 딥러닝 모델을 만들고 학습시키기
  - 딥러닝을 위한 모델 만들기와 학습 단계
  - 인공신경망을 학습시켜 보자
- 신경망 학습의 출력값과 정답의 차이를 알아보자
  - 부드러운 최대값: 소프트맥스 함수
  - 원-핫 인코딩과 범주형 데이터의 평균 제곱오차
  - 원-핫 인코딩 데이터의 오차를 구하는 교차 엔트로피 오차
  - 최적화 기법

01 딥러닝을 위한 도구: 텐서플로

- 텐서플로와 MNIST 데이터
  - **텐서플로**Tensorflow 는 구글이 개발하고 오픈 소스로 제공하는 머신러닝과 딥러닝을 위한 라이브러리이다.
  - 이 라이브러리는 머신러닝 모델을 설계, 훈련, 배포하는데 사용되는데, 딥러닝을 위한 다양한 API와 데이터까지 제공하기 때문에 초보자부터 전문가까지 모두 사용할 수 있다.
  - 텐서플로는 다양한 플랫폼에서 실행될 수 있으며, CPU, GPU, TPU 등 다양한 하드웨어를 지원한다.
    - **GPU**는 **그래픽 처리 장치**(Graphics Processing Unit)의 약자이며 컴퓨터 시스템에서 그래픽 연산을 빠르 게 처리하여 결과값을 모니터에 출력하는 연산 장치이다.
    - **TPU**는 **텐서 처리 장치**(Tensor Processing Unit)의 약자로, 구글에서 개발한 신경망 연산을 빠르게 처리하기 위한 통합 회로 장치이다.
  - 텐서플로는 딥러닝, 신경망 그리고 텐서를 활용한 수치 계산을 포함한 광범위한 머신러닝 알고리즘을 지원하며, 강화학습이나 GANs 등 고급 알고리즘도 편리하게 구현할 수 있다.

- 텐서플로와 MNIST 데이터
  - 딥러닝 프레임워크의 핵심적인 기능과 그 장점을 정리하면 다음과 같이 세 가지로 정리할 수 있다.

1. 자동 미분 기능 제공

- 딥러닝 모델의 학습에서는 오차를 최소화하기 위해 모델의 파라미터에 대한 손실 함수의 기울기 계산이 필요하다.
- 딥러닝 프레임워크는 주어진 연산에 대한 기울기를 자동으로 계산해 준다. 이를 통해 복잡한 모델에 대한 미분값도 쉽게 얻을 수 있으며, 이 기울기값은 최적화 알고리즘에 사용된다.

2. 모듈화된 구성 요소 제공

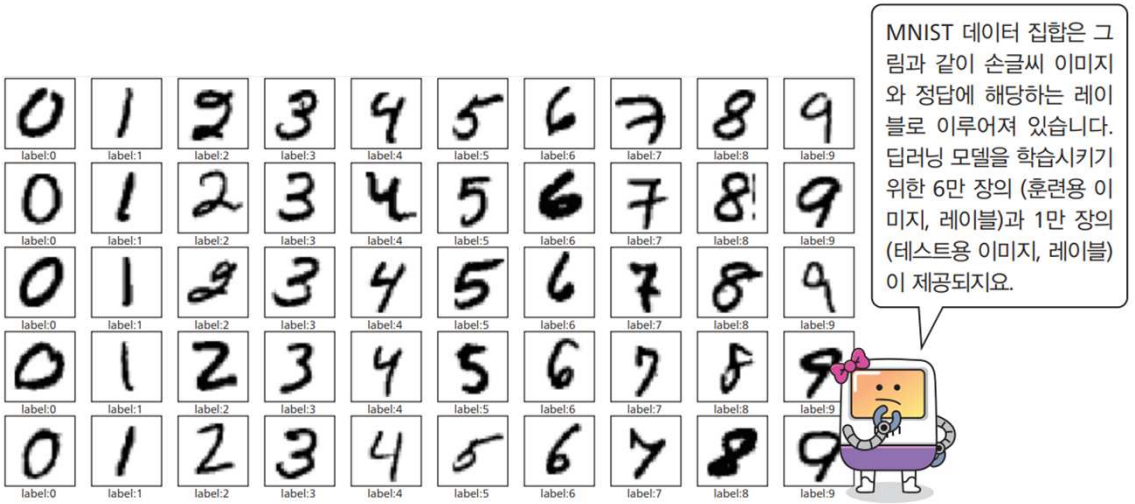
- 대부분의 딥러닝 프레임워크는 인공신경망을 구성하는 기본 구성 요소들을 모듈화하여 제공하고 있다. 이러한 구성 요소들로는 다양한 종류의 퍼셉트론, 활성화 함수, 손실 함수 등이 있다.
- 사용자는 이러한 모듈을 조합하여 복잡한 신경망 구조를 쉽게 설계하고 구현할 수 있다.

3. GPU 지원과 높은 확장성 제공

- 딥러닝 연산, 특히 대규모 데이터 셋을 이용한 심층 신경망의 학습은 많은 연산이 필요하다.
- 딥러닝 프레임워크는 GPU를 활용하여 연산을 빠르게 수행할 수 있으며, 다양한 환경에서의 확장성을 제공한다. 사용자는 CPU와 GPU 간의 전환을 쉽게 할 수 있으며, 복잡한 설정 없이 대량의 데이터를 처리하는 학습을 진행할 수 있다.

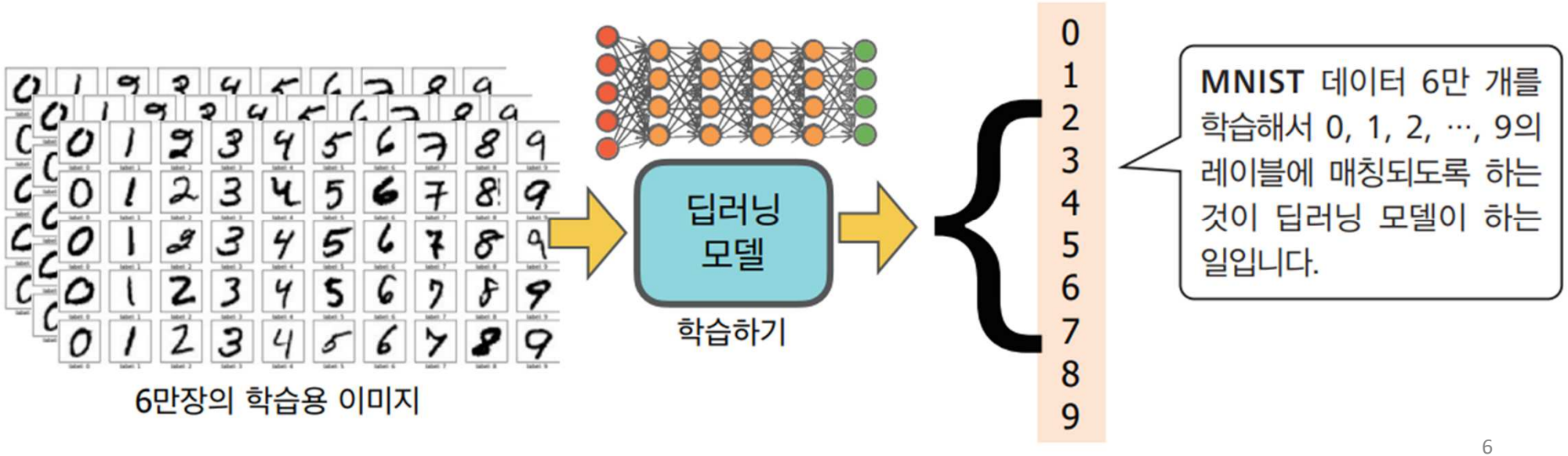
텐서플로와 MNIST 데이터

- 텐서플로는 딥러닝 학습자를 위한 많은 학습 데이터를 제공하고 있는데 이 중 하나가 바로 **MNIST** 이다.
  - MNIST는 Modified National Institute of Standards and Technology의 약어로 미국 인구조사국에서 수집한 손글씨 숫자 데이터 집합을 머신러닝에 적합하도록 수정한 이미지 집합이다.
  - 이 데이터 집합의 숫자 하나는 28×28 픽셀 크기의 회색조 이미지이므로 고성능 컴퓨터가 아니어도 충분히 빠른 처리가 가능하다.
    - 전처리 과정을 통해서 가운데 정렬, 크기 균일화가 잘 이루어져있어서 이미지 처리와 머신러닝에 매우 적합한 데이터이다. 이 이미지 데이터는 케라스 모듈에 포함되어 있다



텐서플로와 MNIST 데이터

- MNIST 데이터 집합은 0부터 9까지 10개의 손글씨 이미지를 각각 6,000장씩 제공한다.
  - 따라서 전체 학습용 데이터는 모두 6만 장의 이미지이며, 테스트를 위한 데이터로 1만 장의 이미지가 별도로 준비되어 있다. 이제 이 데이터를 읽어오는 간단한 프로그램을 작성해 보도록 하자.



텐서플로와 MNIST 데이터

- 텐서플로는 위의 이미지를 제공하는 케라스라는 모듈을 포함하고 있는데, 이 이미지 데이터 집합을 keras.datasets.mnist라는 경로를 통해서 읽어 올 수 있다.
  - 읽어들인 데이터는 이미지, 레이블의 묶음으로 되어 있으며 학습용 이미지와 레이블, 테스트용 이미지와 레이블로 읽어 올 것이다.
- **케라스** Keras 는 딥러닝 모델을 쉽게 만들고 훈련할 수 있도록 설계된 고수준 프로그래밍 인터페이스이다.
- 이 인터페이스는 파이썬 언어로 작성되었으며, 텐서플로, 테아노 혹은 CNTK와 같은 다양한 딥러닝 프레임워크 위에서 실행될 수 있다.
- 이 책의 딥러닝 모델을 생성할 때는 케라스 모듈을 많이 사용하게 될 것이다.

텐서플로와 MNIST 데이터

- MNIST 데이터를 읽어오는 프로그램

```
# tensorflow와 tf.keras를 임포트
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt

# MNIST 데이터를 학습용, 테스트 데이터로 구분하여 읽어 옴
mnist = keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) =\
mnist.load_data()
```

```
Downloading data from
https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11493376/11490434 [==============================] - 0s 0us/step
11501568/11490434 [==============================] - 0s 0us/step
```

텐서플로와 MNIST 데이터

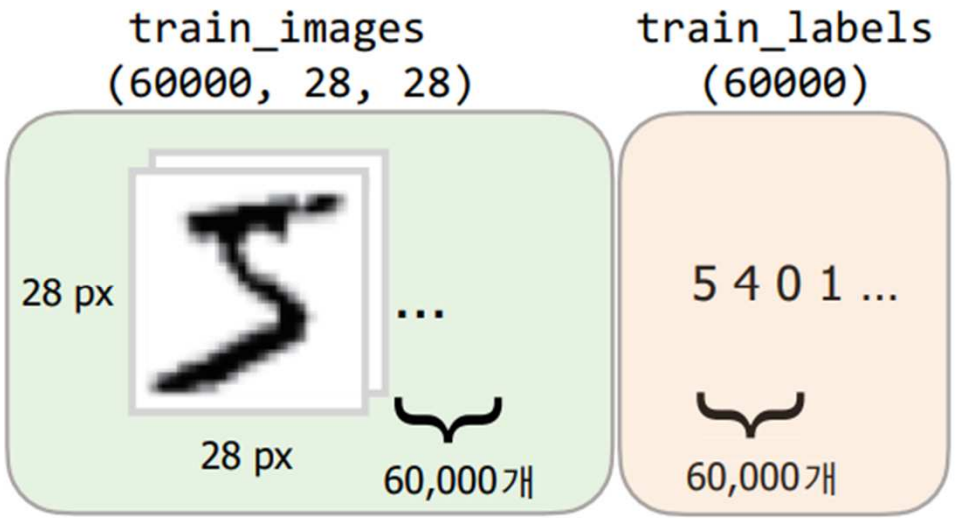
- 읽어들인 데이터가 어떠한 형태인지 다음과 같이 확인해 보자. 각 데이터의 수와 크기를 다음과 같은 코드로 알 수 있다.

```
print('train_images.shape =', train_images.shape)
print('test_images.shape =', test_images.shape)
print('train_labels.shape =', train_labels.shape)
print('test_labels.shape =', test_labels.shape)
```

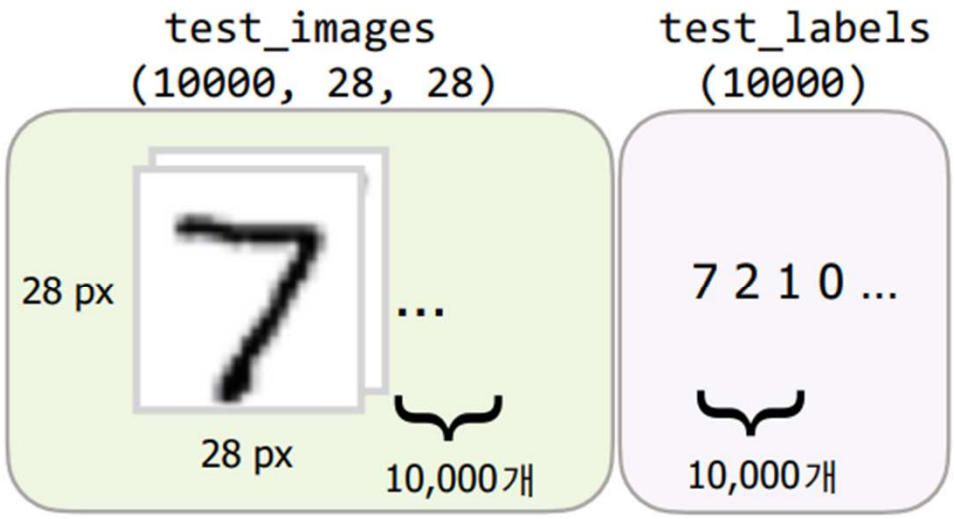
```
train_images.shape = (60000, 28, 28)
test_images.shape = (10000, 28, 28)
train_labels.shape = (60000,)
test_labels.shape = (10000,)
```

텐서플로와 MNIST 데이터

- 조금 더 구체적으로 살펴보면 mnist.load_data() 명령을 통해서 읽어들인 train_images, train_labels, test_images, test_labels는 각각 다음 그림과 같은 모양을 하고 있다.



학습을 위한 6만 개의 이미지와 6만 개의 레이블



테스트를 위한 1만 개의 이미지와 1만 개의 레이블

텐서플로와 MNIST 데이터

- 읽어들인 이미지 데이터를 시각화하기 위하여 다음과 같이 train_images[0]을 이용해서 훈련 이미지의 가장 앞에 있는 데이터를 가져오도록 하자.
- 그리고 이 데이터를 2중 for 문을 사용하여 내부 의 값을 출력하도록 하자.

```
num = train_images[0]  # 첫 번째 데이터 가져오기
for i in range(28):
    for j in range(28):
        print('{:4d}'.format(num[i][j]), end='')
    print()
```

```
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0   0  18  18  18 126 136 175  26 166 255 247 127   0   0
   0   0   0   0   0   0   0   0  30  36  94 154 170 253 253 253 253 253 225 172 253 242 195  64   0   0
   0   0   0   0   0   0   0  49 238 253 253 253 253 253 253 253 253 251  93  82  82  56  39   0   0   0
   0   0   0   0   0   0   0  18 219 253 253 253 253 253 198 182 247 241   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0  80 156 107 253 253 205  11   0  40 154   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0  14   1 154 253  90   0   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0 139 253 190   2   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0  11 190 253  70   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0  35 241 225 160 108   1   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0   0  81 240 253 253 119  25   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0  45 186 253 253 150  27   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0  16  93 252 253 187   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0 249 253 249  64   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0  46 130 183 253 253 207   2   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0   0  39 148 229 253 253 253 250 182   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0  24 114 221 253 253 253 253 201  78   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0  23  66 213 253 253 253 253 198  81   2   0   0   0   0   0   0   0   0
   0   0   0   0   0   0  18 171 219 253 253 253 253 195  80   9   0   0   0   0   0   0   0   0   0   0
   0   0   0   0  55 172 226 253 253 253 253 244 133  11   0   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0 136 253 253 253 212 135 132  16   0   0   0   0   0   0   0   0   0   0   0   0   0   0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
```

텐서플로와 MNIST 데이터

- 2중 for 문을 사용하여 내부를 살펴보면 그 값은 0에서 255로 되어있다.
  - 출력 텍스트의 구조를 조금 자세히 살펴보면, 이 텍스트는 그림과 같이 숫자 5와 유사한 형태를 띄고 있다.
- 이를 좀 더 자세히 살펴보기 위해서 그림과 같이 출력 포맷을 수정해 보자.

```
num = train_images[0]  # 첫 번째 데이터 가져오기
for i in range(28):
    for j in range(28):
        print('{:3d}'.format(num[i][j]), end='')
    print()
```

```
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0 18 18 18 126136175 26166255247127  0  0  0  0
  0  0  0  0  0  0  0  0 30 36 94154170253253253253253225172253242195 64  0  0  0  0
  0  0  0  0  0  0  0 49238253253253253253253253251 93 82 82 56 39  0  0  0  0  0
  0  0  0  0  0  0  0 18219253253253253253198182247241  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0 80156107253253205 11  0 40154  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0 14  1154253 90  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0139253190  2  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0 11190253 70  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0 35241225160108  1  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0 81240253253119 25  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0 45186253253150 27  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0 16 93252253187  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0249253249 64  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0 46130183253253207  2  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0 39148229253253253250182  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0 24114221253253253253201 78  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0 23 66213253253253253198 81  2  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0 18171219253253253253195 80  9  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0 55172226253253253253244133 11  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0 136253253253212135132 16  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
```

텐서플로와 MNIST 데이터

- 이와 같은 코드로 데이터의 형태를 조금 더 자세히 알 수 있다.
  - 이제 이 데이터의 정답값을 확인해 보는 코드를 작성해 보자.
- 이 코드는 다음과 같이 train_labels[0]을 출력해 보면 된다.

```
print('train_labels[0] =', train_labels[0])
```

```
train_labels[0] = 5
```

텐서플로와 MNIST 데이터

- 다음 도전 문제를 풀어보자

도전 문제 7.1: train_images[1] 시각화 하기 상 중 하



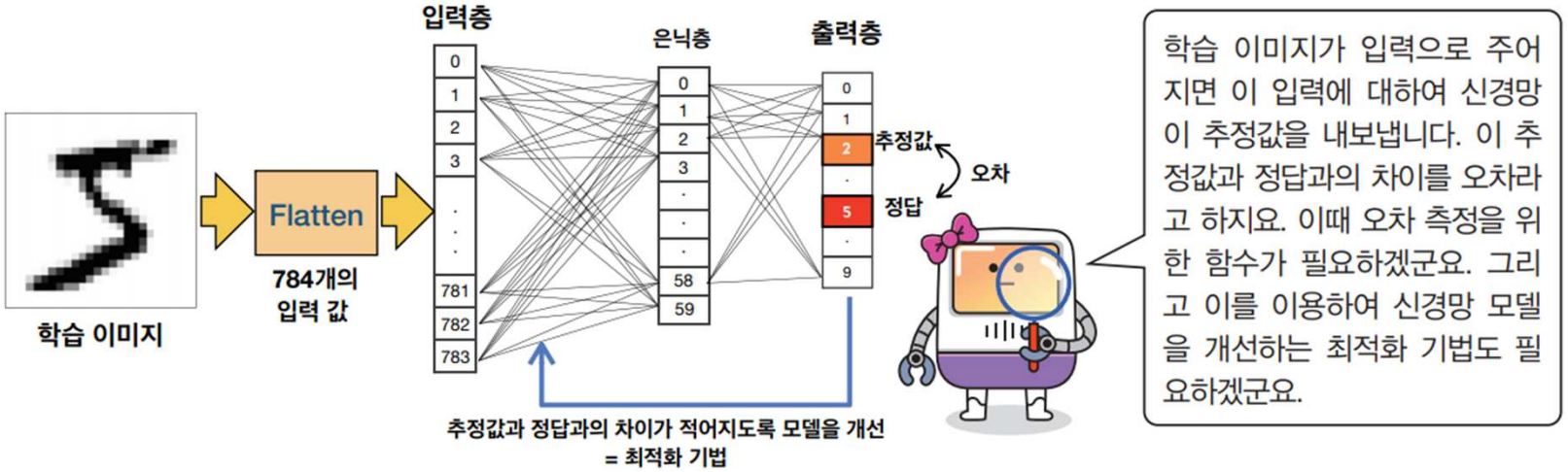
1. 학습용 이미지 집합의 두 번째 항목인 `train_images[1]`을 다음과 같이 시각화하여라.

```
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0 51159253159 50  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0 48238252252252237  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0 542272532523923325257  6  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0 10 60224252253252202 84252253122  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0 0163252252252253252252 96189253167  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0 5123825325319011425322847 79255168  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0 48238252252179 12 75121 21  0  0253243 50  0  0  0  0  0
  0  0  0  0  0  0  0  0  0 38165253233208 84  0  0  0  0  0  0253252165  0  0  0  0  0
  0  0  0  0  0  0  0  0  7178252240 71 19 28  0  0  0  0  0  0253252195  0  0  0  0  0
  0  0  0  0  0  0  0  0 57252252 63  0  0  0  0  0  0  0  0  0253252195  0  0  0  0  0
  0  0  0  0  0  0  0  0 0198253190  0  0  0  0  0  0  0  0  0  0255253196  0  0  0  0  0
  0  0  0  0  0  0  0 76246252112  0  0  0  0  0  0  0  0  0  0253252148  0  0  0  0  0
  0  0  0  0  0  0  0 85252230 25  0  0  0  0  0  0  0  0  7135253186 12  0  0  0  0  0
  0  0  0  0  0  0  0 85252223  0  0  0  0  0  0  0  0  7131252225 71  0  0  0  0  0  0
  0  0  0  0  0  0  0 85252145  0  0  0  0  0  0  0 48165252173  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0 86253225  0  0  0  0  0  0114238253162  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0 85252249146 48 29 85178225253223167 56  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0 8525225225222921525225225219613 0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0 2819925225225325225223314 5  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0 25128252253252141 37  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
```

2. 학습용 이미지에 대한 레이블 집합의 두 번째 항목인 `train_labels[1]`을 출력하고 그 결과를 확인하여라.

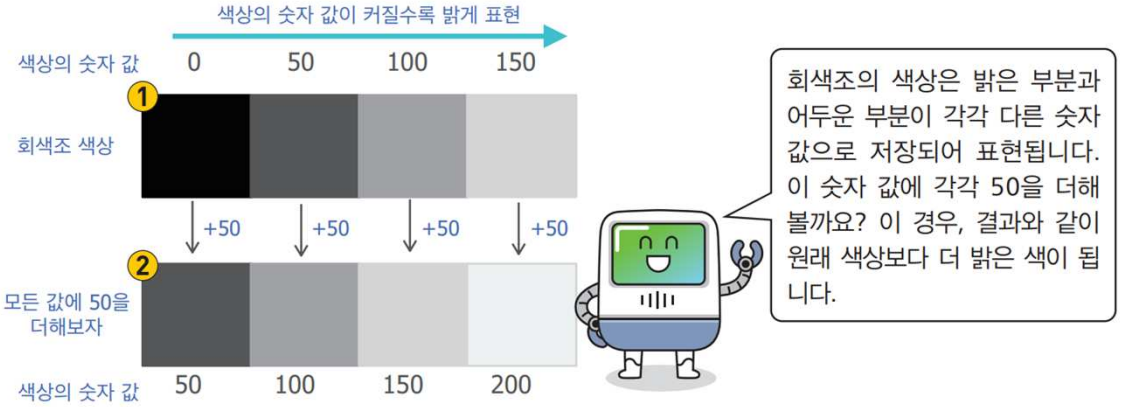
텐서플로와 MNIST 데이터

- 다음 그림과 같이 신경망을 살펴보자.
  - 그림의 신경망은 학습 이미지가 주어져 있고 이 학습 이미지가 입력층의 값으로 주어진다. 이 값은 은닉층을 통과한 후 출력층의 10개 값들 중 하나로 매칭되는 구조를 가지고 있다.
  - 이 신경망에서 학습 이미지에 대한 신경망의 추정값이 5가 아닌 2가 출력된다면 어떻게 해야 할까?
    - 신경망의 가중치를 수정하여 신경망 모델을 개선시켜야만 할 것이다.
  - 이러한 최적화가 바로 신경망의 학습이다. 이를 위하여 우리는 컴퓨터가 다루는 이미지의 구조에 대하여 우선 살펴보도록 하자



컴퓨터는 이미지를 어떻게 저장할까

- 컴퓨터 화면에서 우리가 보는 다양한 모습의 이미지는 컴퓨터 내부에 모두 숫자 값으로 저장되어 있다. 따라서 다음과 같은 이미지는 밝은 부분과 어두운 부분이 각각 다른 숫자 값으로 저장되어 표현된다.
  - 0에서 255까 지의 회색조 이미지의 경우 0에 해당하는 값이 가장 어둡고, 255에 해당하는 값이 가장 밝게 나타난다. 숫자 값이 0, 50, 100, 150일 경우, 이 값을 회색조 색상으로 표현하면 그림 1 과 같이 나타난다.
  - 즉, 색상의 숫자 값이 커질수록 밝게 표현되는 것을 볼 수 있다. 이제 이 숫자 값에 모두 50을 더해 보자. 그렇게 하면 원래 값에서 각각 50이 커진 50, 100, 150, 200이 된다. 이 값을 가지는 회색조 색상 을 화면에 표시하면 그림 2 와 같이 1 에 비해서 한 단계씩 밝게 표현된 이미지를 볼 수 있다



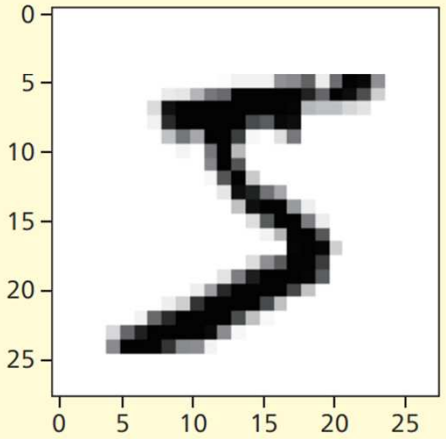
MNIST 데이터 시각화하기

- 앞서 살펴본 train_images 배열의 첫 원소인 train_images[0]을 회색조 이미지로 시각화하도록 하자.
- 이를 위하여 파이썬의 matplotlib이라는 패키지를 import하도록 하자. 이 패키지의 pyplot 서브 패키지에 있는 imshow() 함수는 배열값을 이미지로 보여주는 함수이다.

```
import matplotlib.pyplot as plt

plt.imshow(train_images[0], cmap='Greys')
```

```
<matplotlib.figure.AxesImage at 0x7efe51435b50>
```

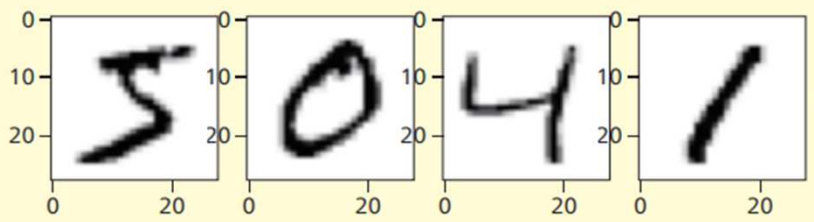


MNIST 데이터 시각화하기

- 텐서플로의 케라스 패키지의 MNIST 데이터 셋에는 이러한 학습용 데이터가 모두 6만 개나 제공된다. 하지만 이를 모두 보여줄 수 없으므로 가장 먼저 나타나는 네 개의 이미지를 가져와서 화면에 출력해 보자.
- 네 개의 이미지를 한 화면에 보여주기 위하여 다음과 같이 맷플롯립의 add_subplots() 함수를 사용하였다.

```
fig = plt.figure()
ax1 = fig.add_subplot(1, 4, 1)
ax2 = fig.add_subplot(1, 4, 2)
ax3 = fig.add_subplot(1, 4, 3)
ax4 = fig.add_subplot(1, 4, 4)

ax1.imshow(train_images[0], cmap='Greys') # 첫 번째 학습용 데이터
ax2.imshow(train_images[1], cmap='Greys') # 두 번째 학습용 데이터
ax3.imshow(train_images[2], cmap='Greys') # 세 번째 학습용 데이터
ax4.imshow(train_images[3], cmap='Greys') # 네 번째 학습용 데이터
```



MNIST 데이터 셋의 train_images 데이터

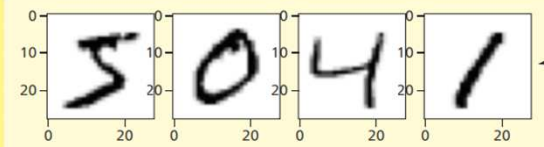
MNIST 데이터 셋의 첫 번째, 두 번째, 세 번째, 네 번째 데이터입니다. 이 데이터에 대하여 각각 5 0 4 1의 레이블(정답)이 제공됩니다.

MNIST 데이터 시각화하기

- 이 결과를 통해서 이미지 파일 형식의 학습 데이터들에서 가장 앞에 나타나는 네 개 데이터의 형태가 5, 0, 4, 1의 숫자라는 것을 눈으로 확인할 수 있다.
- 이제 이 네 개 이미지의 정답에 해당하는 train_labels 배열의 값을 출력해 보도록 하자.

```
print('train_labels[:4] =', train_labels[:4])
```

```
train_labels[:4] = [5 0 4 1]
```



MNIST 데이터 셋의 train_images 데이터

MNIST 데이터 셋의 첫 번째, 두 번째, 세 번째, 네 번째 데이터입니다. 이 데이터에 대하여 각각 5 0 4 1의 레이블(정답)이 제공됩니다.

- 출력 결과, 훈련 이미지의 정답에 해당하는 레이블이 5 0 4 1이라는 것을 확인할 수 있었다.
- 이런 형 태의 이미지와 정답이 6만 개 주어져 있는 것이 MNIST 데이터이므로 우리는 이 필기체 이미지를 학습하여 정답을 알아맞히는 딥러닝 모델을 만들어야 할 것이다.

MNIST 데이터 시각화하기

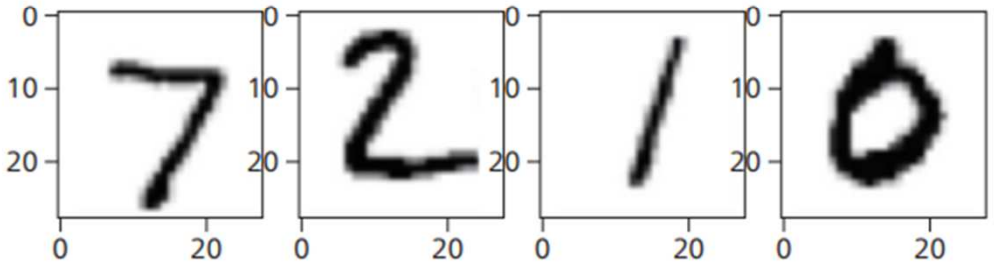
- 다음 도전 문제를 풀어보자

도전 문제 7.2: 테스트용 이미지 시각화하기

상 중 하



1. 테스트용 이미지 집합의 첫 번째 항목부터 네 번째 항목의 이미지를 다음과 같이 시각화하여라.



2. 테스트용 이미지에 대한 레이블 집합인 `test_labels`의 첫 항목부터 네 번째 항목의 값을 다음과 같이 출력하여라.

```
테스트 이미지의 레이블들: [7 2 1 0]
```

02 딥러닝 모델을 만들고 학습시키기

- 딥러닝을 위한 모델 만들기와 학습 단계
  - 이전 절에서 살펴본 MNIST 데이터 셋 이미지를 구분하는 딥러닝 모델을 만들어 보자. 이 모델을 만들기 위하여 필요한 단계를 다음과 같이 1에서 6까지의 단계로 나누어 보았다
  - 보기에는 복잡해 보이지만 케라스 서브 모듈의 다양한 기능을 사용한다면 이런 어려운 작업도 손쉽게 할 수 있다. 이러한 작업은 이전 장에서 배운 다층 퍼셉트론을 이용하여 구현할 수 있는데, **텐서플로는 다층 퍼셉트론을 만들고 학습시키는 복잡한 작업을 매우 쉽게 할 수 있다**.

1. 신경망 모델의 입력값으로 사용하기에 적절한 형태로 `train_images` **데이터를 전처리**한다.
2. `train_images` 데이터를 학습시키기 위한 **신경망 모델을 정의**한다.
3. 신경망 모델의 **활성화 함수, 학습을 위한 최적화 함수, 손실 함수, 측정 방법을 정의**한다.
4. 6만 개의 이미지로 이루어진 `train_images` 데이터를 심층 신경망 모델에 넣어서 `train_labels` 데이터의 숫자로 인식하도록 **학습을 시킨다**.
5. 1만 개의 이미지로 이루어진 `test_image` 데이터를 학습을 마친 모델의 입력으로 넣어서 이 모델의 **정확도를 알아본다**.
6. 생성된 모델의 성능을 점차 개선시킨다.

딥러닝을 위한 모델 만들기와 학습 단계

- 1단계: 데이터 전처리
  - 신경망 모델의 입력값으로 사용하기에 적절하도록 만드는 전처리는 이미 MNIST 데이터 집합을 생성할 적에 실행되었다고 볼 수 있다.
    - 만일 여러분이 직접 사람의 손글씨 이미지 데이터를 수집한다면, 손글씨 데이터를 스캐닝하는 과정에서 상당한 잡음 이미지가 들어갈 수 있을 것이다.
    - 이 경우에 잡음을 제거하거나 크기를 보정하는 작업이 될 수 있다.
    - 이러한 작업은 현재는 불필요한 것으로 보이지만 실데이터를 다룬다면 매우 중요한 작업이 될 것이다. (MNIST 데이터는 미리 전처리를 한 데이터이다)
    - 여기에서는 0에서 255 사이값을 가지는 회색조 이미지의 값의 범위를 0에서 1 사이의 값으로 변환시키는 작업이 필요하다.

딥러닝을 위한 모델 만들기와 학습 단계

- 1단계: 데이터 전처리
  - 이러한 전처리는 train_images / 255, test_images / 255로 간단하게 할 수 있다.
  - 이 전처리는 **원래 픽셀이 가진 값의 범위 0~255를 0에서 1 사이의 값으로 조정**하는 **정규화**라고도 할 수 있다.
  - 이와 같이 0과 1 사이의 실수값을 입력으로 사용하는 이유는 인공신경망의 전체 수식에서 사용되 는 값이 0, 1, 2, 3과 같은 이산적인 값이 아닌 연속적인 값이어서 **실수 값을 입력으로 사용하는 것이 더 나은 결과를 얻을 수 있기 때문**이다.

```
# 입력값 전처리: 0에서 1 사이의 값으로 입력 데이터를 가공함
train_images, test_images = train_images / 255, test_images / 255
```

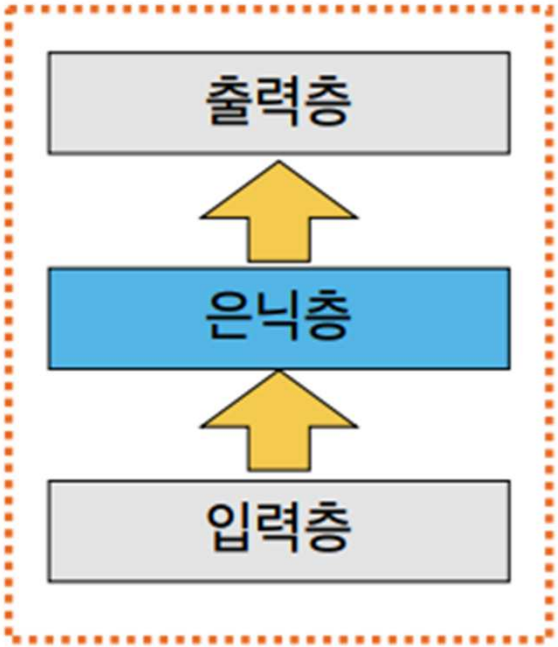
딥러닝을 위한 모델 만들기와 학습 단계

- 2단계: 신경망 모델 만들기
  - 신경망 모델을 만드는 복잡한 일을 쉽게 할 수 있도록 도와주는 것이 텐서플로의 역할이다.
  - 우리가 흔히 이야기하는 **딥러닝** deep learning 이라는 것은 인공신경망의 **층** layer 을 **계층적으로 깊이 쌓아서 만든 신경망 모델을 통해서 학습을 하는 알고리즘**을 말한다.
  - 다층 퍼셉트론을 만들고 오차 역전파 알고리즘을 통해서 학습을 수행하도록 하기 위해서는 대규모의 행렬 계산이 필요한데, 이러한 복잡한 작업은 아래와 같은 간단한 텐서플로 코드로 구현할 수 있다.

```
# 순차적 연결 구조의 신경망 모델을 만든다
# 순차적 연결 구조의 신경망 생성 시 입력값은 리스트 형식으로 제공된다
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(60, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])
```

딥러닝을 위한 모델 만들기와 학습 단계

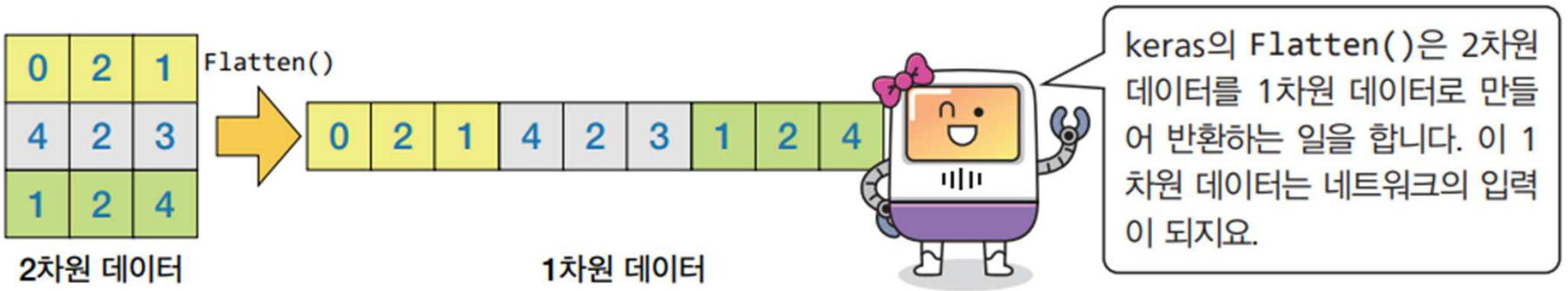
- 2단계: 신경망 모델 만들기
  - 위의 코드에서 keras.Sequential 클래스는 그림과 같이 입력층, 은닉층, 출력층이 존재하는 **순차적 연결 구조를 가지는 신경망 모델을 생성**한다.
  - 이 순차적 신경망 모델을 생성할 때는 리스트 형식의 인자를 받는데, 이 리스트의 첫 항목은 일반적으로 입력층이 된다.



keras의 순차적 신경망 구조

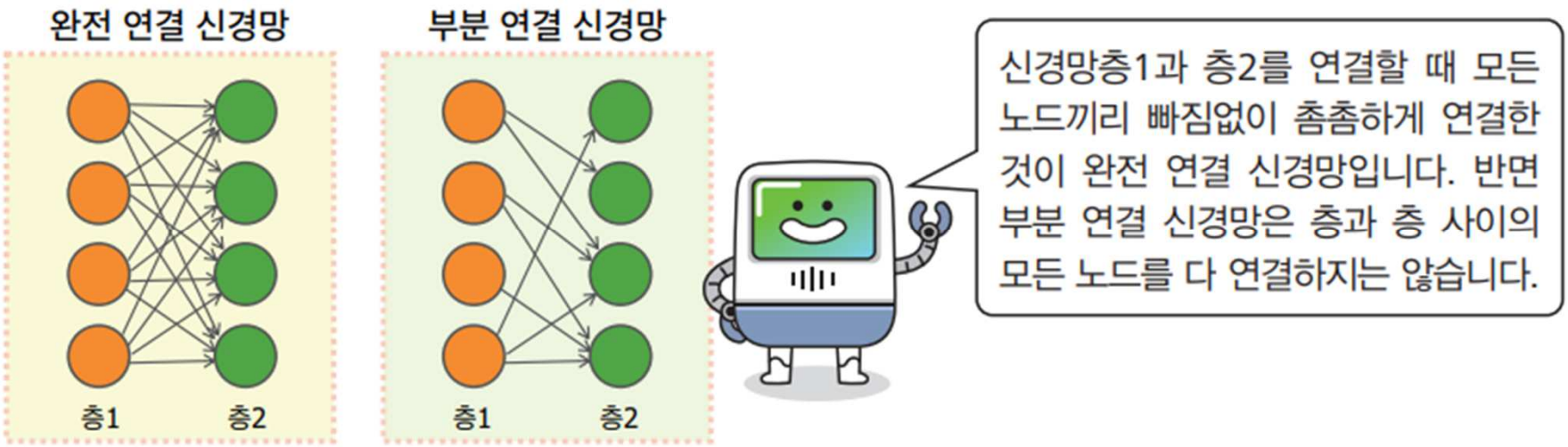
딥러닝을 위한 모델 만들기와 학습 단계

- 2단계: 신경망 모델 만들기
  - 우리가 다룰 순차적 심층 신경망은 입력값으로 1차원 배열을 사용한다. 하지만 입력 이미지는 2차원 배열 형태로 되어있기 때문에 이를 Flatten 클래스를 사용하여 1차원 객체로 만든다.
  - 2차원 이미지를 1차원 배열로 구성하면 이미지의 기하학적 정보가 손실되는데, 이에 관한 자세한 내용은 이후의 **합성곱 신경망** convolutional neural network: CNN 에서 상세히 다룰 것이다.
  - Flatten 네트워크에서는 2차원 입력을 1차원 값으로 만들기 때문에 필요한 매개변수는 입력의 크기이다. 이 경우는 28 × 28 = 784가 입력 노드의 수이다.



딥러닝을 위한 모델 만들기와 학습 단계

- 2단계: 신경망 모델 만들기
  - 다음으로 두 층에 대한 촘촘한 연결을 구성하는 Dense 네트워크가 나타난다.
  - Dense는 학습을 위한 연결을 **밀집된** dense 구조 혹은 **완전 연결** fully connected 층으로 한다는 의미이다.
  - 그림에서 살펴보면 층1과 층2에 각각 네 개의 노드가 있기 때문에 완전 연결 층의 모든 노드를 연결하는 매개변수는 모두 16개이다. 반면 **부분 연결 신경망**에서는 층과 층 사이의 모든 노드를 다 연결하지는 않는다.



딥러닝을 위한 모델 만들기와 학습 단계

- 2단계: 신경망 모델 만들기

  - Dense 네트워크의 입력은 앞 층에서 주어지기 때문에 몇 개의 출력으로 연결할지를 정하는 매개변수가 있다. 여기에서는 60개의 노드로 네트워크를 구성하므로 784 × 60 = 47,040개의 연결 네트워크가 생성된다(**실제 생성되는 내부의 은닉층은 60개의 노드와 편향 노드가 추가되어 61개의 노드를 가진다**).

  - 이와 같이 1차원 배열로 변환된 784개의 값은 은닉층을 가진 순차적인 신경망을 통과하여 10개 중 하나의 **범주**category 로 분류된다.
  - 이때 은닉층을 포함한 학습층을 만드는 명령이 keras.layers. Dense()이다. 이렇게 생성된 신경망은 이전에 다룬 **1) 경사하강법을 이용하여 오차를 줄이는 방향 으로 학습을 진행**하며, **2) 오차 역전파를 이용하여 가장 좋은 가중치와 편향값**을 찾아 나간다.

딥러닝을 위한 모델 만들기와 학습 단계

- 2단계: 신경망 모델 만들기
  - 이 모델의 구조를 model.summary() 메소드를 통해서 살펴보자.
    - 784개의 입력이 Flatten 타입으로 들어옴
    - 중간에 61개의 은닉 노드가 출력층에 있는 10개의 노드와 연결된 구조임을 알 수 있다.
  - 이 모델에서 학습시켜야 할 **파라미터** parameter 의 수는 47,710개라는 정보도 함께 보여 준다.
  - 앞 장에서 설명한 바와 같이 퍼셉트론의 노드를 연결하는 과정에서 외부 자극을 흉내내는 특정한 숫자가 곱해지는데, 이 곱해지는 값을 **가중치**, **파라미터** 또는 **매개변수**라고 부른다.

```
model.summary()    # 모델의 구조를 요약하여 살펴보자
```

```
Model: "sequential_10"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 flatten_7 (Flatten)         (None, 784)               0

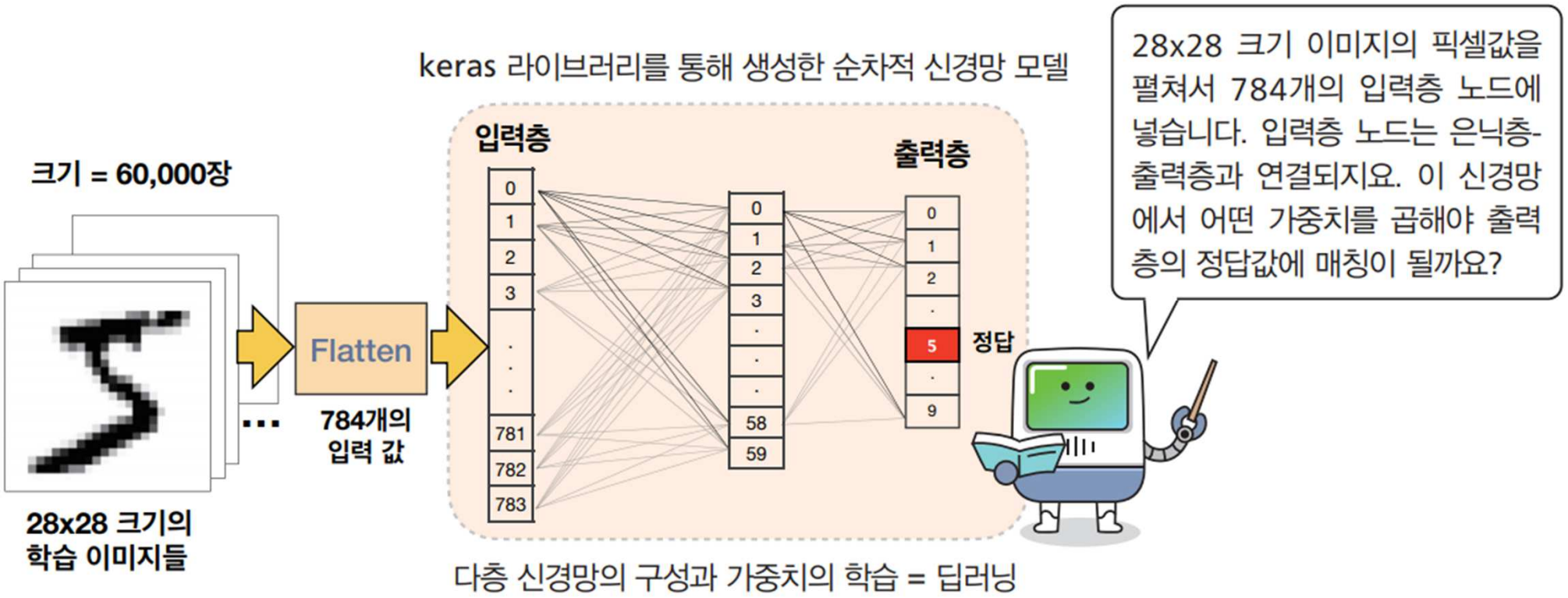
 dense_22 (Dense)            (None, 60)                47100

 dense_23 (Dense)            (None, 10)                610

=================================================================
Total params: 47,710
Trainable params: 47,710
Non-trainable params: 0
_________________________________________________________________
```

딥러닝을 위한 모델 만들기와 학습 단계

- 2단계: 신경망 모델 만들기



인공신경망을 학습시켜 보자

- 3단계: 신경망 모델의 활성화 함수, 최적화 함수, 측정 방법 정의

  - 앞에서 만든 모델은 아직 데이터를 가지고 학습한 상태가 아니다.
  - 신경망의 학습은 기본적으로 추측을 한 뒤에 정답과 비교하여 오차가 얼마인지 확인한 후, 이 오차를 줄이는 방법으로 연결의 강도를 조절하는 것이다.
  - 위의 그림을 바탕으로 설명하자면 학습 이미지가 입력으로 주어졌을 때, 이 입력에 대하여 신경망이 추정값 5를 내보냈다.
    - 이 값은 정답 5와 같다. 만일 추정값으로 5가 아닌 2가 나타날 경우, 정답과 추정값의 차이가 나며 이 차이가 바로 오차이다.
  - 이 오차 측정을 위한 함수가 필요할 것이며, 이 오차를 이용하여 신경망 모델을 개선하는 최적화 기법도 필요할 것이다

인공신경망을 학습시켜 보자

- 3단계: 신경망 모델의 활성화 함수, 최적화 함수, 측정 방법 정의
  - 신경망 모델을 학습시키기 위해서는 데이터와 함께, 순차적인 구조 그리고 오차를 측정하는 방법과 이 오차를 줄이는 방법을 지정해야만 한다.
  - 이것을 하는 것이 **3번 단계**이다.
    - 활성화 함수, 학습을 위한 최적화 함수, 손실 함수, 오차 측정 방법 등에 관한 내용은 모델을 학습시키기 전에 필요하며 다음과 같은 설정을 하면 된다.

```
model.compile(optimizer='adam',\
              loss='sparse_categorical_crossentropy',\
              metrics=['accuracy'])
```

인공신경망을 학습시켜 보자

- 3단계: 신경망 모델의 활성화 함수, 최적화 함수, 측정 방법 정의
  - 모델을 점점 더 좋은 상태로 만드는 것을 **최적화**optimizer 라고 부른다.
  - 최적화를 위해서는 현재 모델이 얼마나 잘못되었는지 알아야 한다.
  - 현재 모델의 정답과 실제 정답의 차이가 오차이고 이 오차를 측정하는 것이 **손실 함수**이다.
    - 손실 함수로 무엇을 사용할 것인지 지정해야 하는데, 여기서는 **sparse_categorical_crossentropy()** 함수를 사용했다.
  - 손실 함수를 이용하여 모델을 개선하는 최적화 기법이 필요함
    - **optimizer='adam'**을 통해서 **Adam 최적화 알고리즘**을 선택하였다.
    - Adam 최적화 알고리즘은 경사하강법의 한 종류이다.
  - 다음으로 **metrics=['accuracy']**가 하는 일을 살펴보자.
    - Metric이란 척도라는 뜻으로 이 **모델이 학습을 통해서 목표를 얼마나 잘 달성하였는지 나타내는 값**이다.
    - accuracy를 선택하였는데 이는 (모델이 정확하게 예측한 데이터)/(전체 데이터)를 목표 달성의 척도로 삼겠다는 의미이다.
    - 따라서 전체 데이터 6만 개에 대하여 모델이 6만 개를 모두 정확하게 예측하면 1.00이 되며, 6만 개 중에서 5만 4천 개를 정확하게 예측하면 0.9가 된다.

인공신경망을 학습시켜 보자

- 4단계: 학습용 데이터로 모델을 학습시키기

  - 모델이 완성되면 **학습용 데이터와 정답 데이터(레이블)를 주고 학습을 실시**하면 된다.
  - **4번 단계**인 학습을 시작하게 하는 함수는 모델의 **fit()** 메소드인데, 학습용 입력과 여기에 대응하는 정답 레이블 데이터 셋을 차례로 주면 된다.
  - 학습 데이터 모음을 가지고 한 번 훈련을 실시하는 것을 **에폭**epoch 이라고 부르는데, 이 과정을 반복함으로써 모델은 더 좋은 상태로 바뀌게 된다.
  - 반복 훈련은 이전 절에서 다룬 오차 역전파 알고리즘에 의해서 이루어지는데, 이 모델은 오차 역전파에 의해서 오차를 줄여 나가는 학습을 진행한다.

인공신경망을 학습시켜 보자

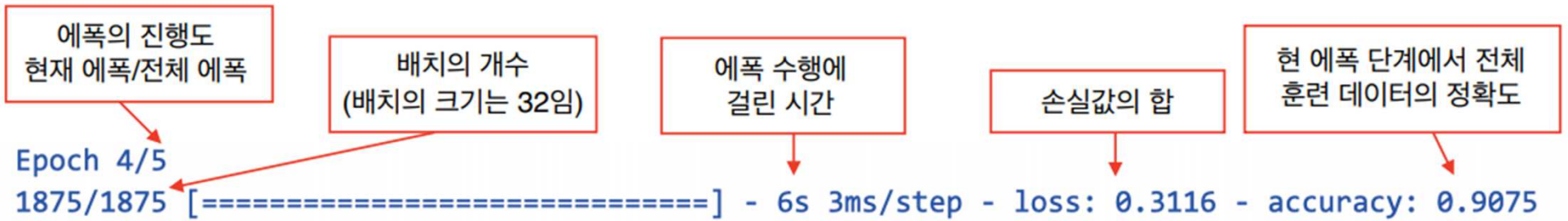
- 4단계: 학습용 데이터로 모델을 학습시키기
  - 여기서는 **epochs=3**이라는 키워드 인자를 통해 에폭을 3으로 지정하였으므로 **전체 데이터에 대한 학습을 세 차례 실시**하게 된다.

```
model.fit(train_images, train_labels, epochs=3)
```

```
Epoch 1/3
1875/1875 [==============================] - 7s 3ms/step - loss:
0.3022 - accuracy: 0.9143
Epoch 2/3
1875/1875 [==============================] - 5s 2ms/step - loss:
0.1475 - accuracy: 0.9577
Epoch 3/3
1875/1875 [==============================] - 5s 2ms/step - loss:
0.1068 - accuracy: 0.9696
<keras.callbacks.History at 0x7f5f40b4c400>
```

인공신경망을 학습시켜 보자

- 4단계: 학습용 데이터로 모델을 학습시키기
  - 에폭이 진행되면서 매 에폭 단계에서의 손실값 loss와 정확도 accuracy가 화면에 출력된다.
  - 이 값의 변화를 자세하게 살펴보면 훈련이 진행됨에 따라 예측의 부정확도를 파악할 수 있는 손실값 loss는 감소하고, 예측의 정확도를 나타내는 accuracy 값은 증가함을 확인할 수 있다.
  - 학습 단계의 출력값과 그 의미는 다음과 같다.(다음은 에폭이 5인 경우이다)

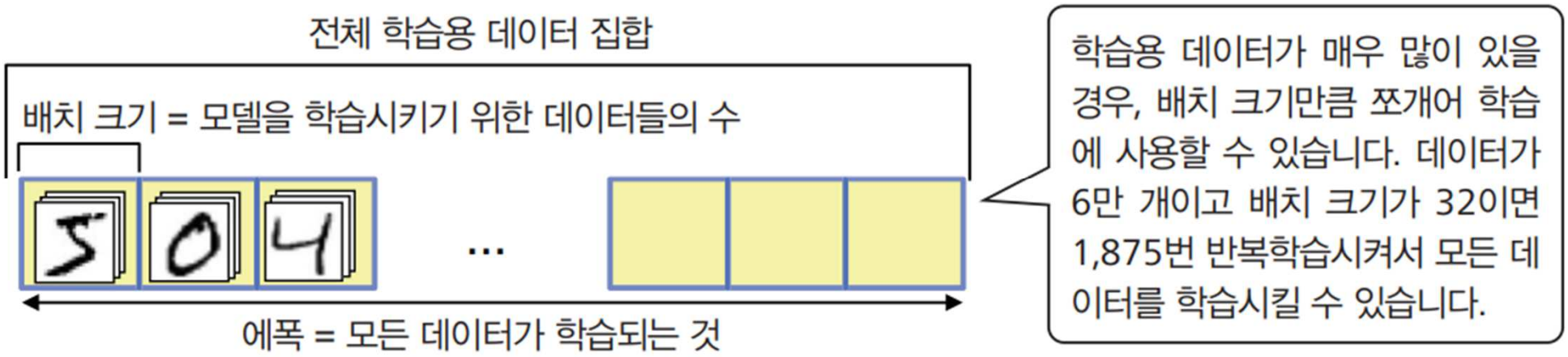


인공신경망을 학습시켜 보자

- 4단계: 학습용 데이터로 모델을 학습시키기
  - 이 설명에서 나타난 **배치**batch 란 무엇일까? 배치의 크기와 에폭에 대해서 다음의 그림을 통해서 알아보도록 하자.
    - 만일 여러분이 딥러닝 모델을 학습시키기 위하여 훈련용 데이터를 모델의 입력으로 넣을 때 이 데이터가 너무 클 경우, 어떤 일이 발생할까 생각해 보자.
    - 예를 들어 위의 MNIST 데이터 6만 개를 모두 메모리에 올려 두고 학습을 시키면 매우 많은 메모리가 필요하며 이를 학습하기 위한 시간도 많이 소요될 것이다.
  - 이 때문에 학습 시에는 전체 데이터를 몇 개의 단위로 나누어서 단위별로 학습을 시키는 것이 효과적일 것이다. 이러한 **학습 데이터의 단위를 배치라고 한다.**

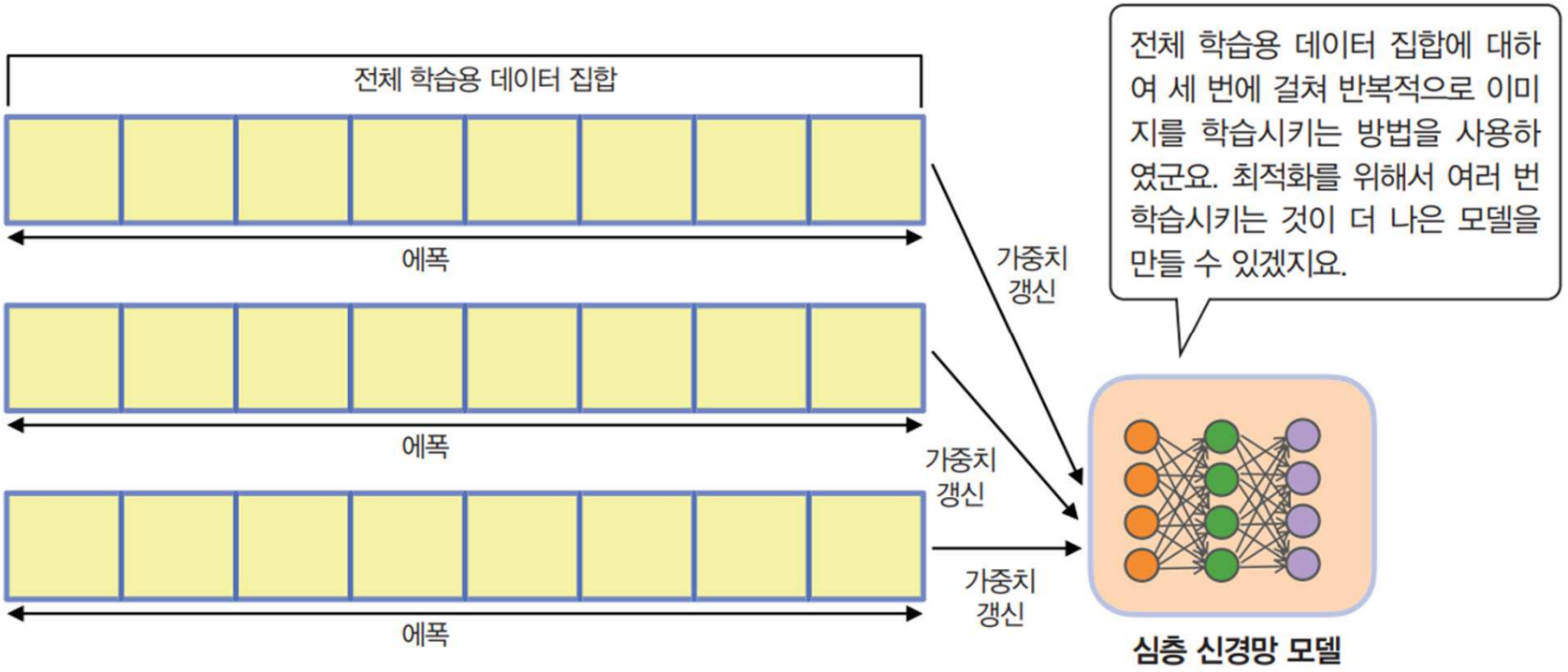
인공신경망을 학습시켜 보자

- 4단계: 학습용 데이터로 모델을 학습시키기
  - 이 모델의 경우에는 디폴트 배치 값인 32개의 이미지를 묶어서 학습을 진행한다. 이 과정을 1,875 번 반복할 경우 $32 \times 1{,}875 = 60{,}000$개의 이미지가 모두 학습된다.
  - 이와 같이 모든 데이터가 학습되 는 것이 1 에폭이다.
    - 예시에서는 학습 단계가 **Epoch 4/5**로 나타나 있는데, **총 5회의 에폭 중에서 현재 4회의 에폭이 진행 중**이라는 의미이다. 배치의 크기는 모델 학습 메소드인 **fit()**의 **batch** 키워드 인자로 설정하는 것도 가능하다.



인공신경망을 학습시켜 보자

- 4단계: 학습용 데이터로 모델을 학습시키기
  - 앞의 모델은 1 에폭 동안 전체 데이터를 학습에 사용하여 4만 7천 개가 넘는 파라미터를 갱신하게된다. 이러한 에폭을 여러 번 하는 이유는 무엇일까?
  - 바로 최적화를 통해서 더 나은 모델을 만들기 위해서는 동일한 학습 데이터라 하더라도 한 번에 그치지 않고 여러 번 학습시키는 것이 더 좋기 때문이다.



인공신경망을 학습시켜 보자

- 다음 도전 문제를 풀어보자

도전 문제 7.3: 배치 크기 수정하기

상 중 하



1. `model.fit(train_images, train_labels, epochs=3)` 코드에 `batch_size=64`라는 키워드 인자를 넣어서 모델을 훈련시키자. 최종 정확도에는 어떠한 변화가 있는가?

2. `epochs=5`와 같이 에폭의 수를 늘려서 모델을 훈련시키자. 최종 정확도에는 어떠한 변화가 있는가?

인공신경망을 학습시켜 보자

- 5단계: 모델의 정확도 알아보기
  - 이제 학습 데이터가 아닌 학습에 사용하지 않았던 테스트 데이터를 이용하여 이 모델이 정답 레이블을 잘 맞히는지 확인해 보자.
    - 1만 개의 이미지로 이루어진 테스트 데이터는 생성한 **모델이 처음 보는 데이터**이다.
  - 따라서 이것을 적용한 결과는 학습용 데이터로 훈련한 최종 결과에 비해 정확도가 조금 떨어질 수 있다. 다음의 경우 정확도 값이 0.7907로 나타났는데, 79.07%의 정확도로 테스트 데이터를 정답으로 예측하였음을 알 수 있다.

```
test_loss, test_acc = model.evaluate(test_images, test_labels,\
                                     verbose=2)
print('\n테스트 정확도:', test_acc)
```

```
313/313 - 0s - loss: 0.6141 - accuracy: 0.7907

테스트 정확도: 0.7907000184059143
```

인공신경망을 학습시켜 보자

- 5단계: 모델의 정확도 알아보기

  - 신경망의 학습 데이터는 동일하지만 학습 때 사용되는 모델의 초기 가중치는 매번 랜덤하게 결정되므로 동일한 모델을 만들더라도 정확도 값은 매번 다르게 나타날 수 있다.
  - **evaluate()** 함수의 인자인 **verbose**는 이 모델의 학습 진행과 테스트의 진행상황을 사용자에게 어떤 방식으로 전해줄 것인지 결정하는 일을 한다. **verbose** 키워드 인자의 값으로는 정수 0, 1, 2를 가질 수 있다. 만일 **verbose** 값이 0이면 침묵 모드가 되어 진행 상황을 보여주지 않으며, **verbose** 값이 1이면 진행도를 막대 줄로 보여준다. 마지막으로 **verbose** 값이 2이면 에폭당 하나의 라인으로 그 결과를 보여준다.
  - 이렇게 층을 쌓아 나가며 여러 개의 은닉층을 가지는 신경망을 얻을 수 있고, 이것을 **심층 신경망** deep neural network 이라고 한다.
    - **심층 신경망에서 학습을 진행하는 것이 딥러닝**의 기본적인 개념이다.

인공신경망을 학습시켜 보자

- 6단계: 모델의 성능 개선시키기

  - 심층 신경망 모델의 학습 결과를 바탕으로 이 모델의 성능을 개선시키는 것이 마지막 단계이다.
  - 모델의 성능을 개선시키기 위해서는 신경망의 수를 증가시키거나, 학습 데이터의 양을 증가시키거나, 에폭을 여러 번 수행하거나, 모델 구성 시에 완전 연결망 대신 부분 연결망 모델을 사용하는 등 여러 가지 방법이 있다.

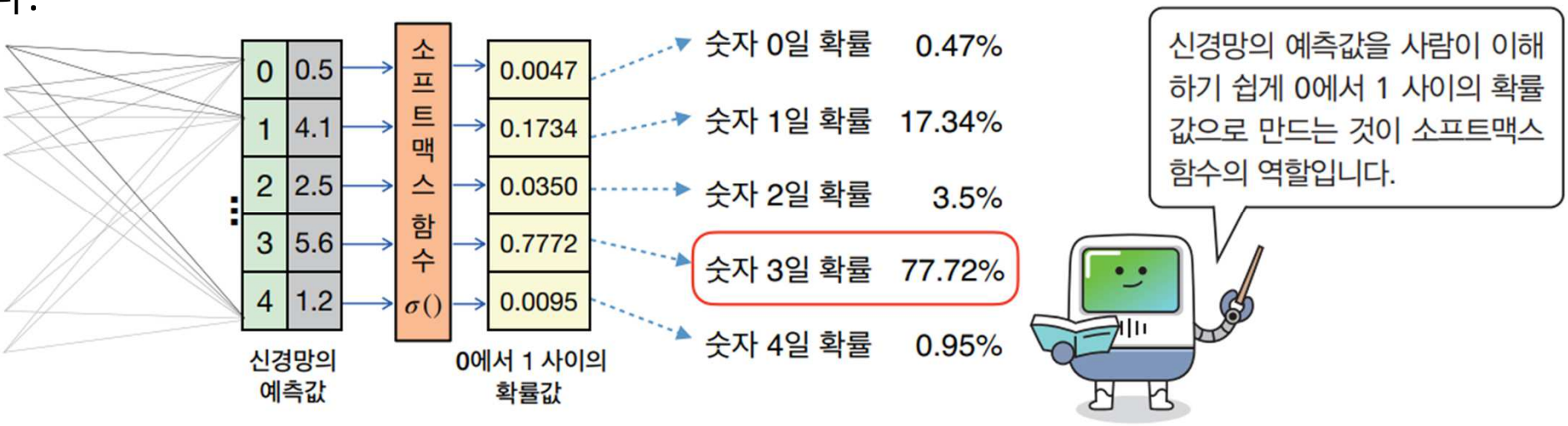
03 신경망 학습의 출력값과 정답의 차이를 알아보자

- 부드러운 최대값: 소프트맥스 함수
  - 우선 가정을 단순히 하기 위하여 아래와 같이 **0에서 4까지 5개의 입력 데이터와 정답 레이블이 있는 경우를 가정하자.**



부드러운 최대값: 소프트맥스 함수

- 이 데이터를 이용한 학습 모델은 모두 5개의 출력 카테고리가 있을 수 있기 때문에 임의의 입력값 에 대하여 최종 노드의 출력은 [0.5, 4.1, 2.5, 5.6, 1.2] 와 같은 예측값이 나올 수 있다.
- 이러한 **예측값을 가진 벡터를 0에서 1 사이의 클래스별 확률값으로 변환**하기 위해서 사용하는 함수가 바로 **소프트맥스**softmax 함수이다.
  - 소프트맥스 함수를 사용하면 **신경망의 예측값을 사람이 이해하기 쉽다는 장점**이 있다.



부드러운 최대값: 소프트맥스 함수

- $i = 1, \ldots, K$이고 $z = (z_1, \ldots, z_K) \in \boldsymbol{R}^K$일 때 소프트맥스 함수를 $\sigma()$라고 하면, 다음과 같이 자연 상수 e를 밑으로 하는 지수함수를 사용하여 정의할 수 있다.

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

- 여기에서 z_i 는 그림 왼쪽에 나타난 **신경망의 예측값**인데, 이 그림의 경우, 모델이 반환하는 z_1, z_2, z_3, z_4, z_5의 값이 각각 $0.5, 4.1, 2.5, 5.6, 1.2$가 된다.
- 따라서 소프트맥스 함수의 분모 값은 $e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4} + e^{z_5}$ 를 계산한 $e^{0.5} + e^{4.1} + e^{2.5} + e^{5.6} + e^{1.2}$이 된다. 그리고 이 분모값에 대하여 $e^{0.5}$을 분자로 하는$e^{0.5} \,/\, (e^{0.5} + e^{4.1} + e^{2.5} + e^{5.6} + e^{1.2})$ 값이 소프트맥스 함수의 첫 번째 출력값 $\sigma(z_1)$이 된다.
  - 여기서는 0.0047 값이 바로 그 값인데, 이 값은 네트워크의 예측값이 숫자 0일 확률(약 0.47%)을 의미 한다.

부드러운 최대값: 소프트맥스 함수

- 이렇게 구한 소프트맥스 함수의 전체 출력은 [0.0047, 0.1734, 0.0350, 0.7772, 0.0095]가 된다.
  - 즉, 이 모델의 예측값을 소프트맥스 함수를 이용하여 각각의 확률로 계산했을 때 **숫자 3일 확률이 77.72%로 가장 높다**고 할 수 있다.
- 이런 특징으로 인해 이 함수는 **부드러운 최대값**을 얻는다는 의미의 소프트맥스라는 이름을 얻게 되었다. 여기서 '부드럽다'는 것은 다른 출력값의 확률값도 유지한다는 뜻이 있다.
- 반면 **max**나 **argmax**와 같은 함수는 각각 최대값 혹은 최대값을 갖게 하는 **인자**argument 하나만을 반환한다.

부드러운 최대값: 소프트맥스 함수

- 이상의 과정은 다음과 같이 넘파이를 이용하여 구현할 수 있다.

```
import numpy as np

def softmax(a):
    exp_of_a = np.exp(a)                # 각각의 지수함수
    sum_exp = np.sum(exp_of_a)          # 지수함수값의 합
    y = exp_of_a / sum_exp              # 각각의 확률값을 구함
    return y

a = np.array([0.5, 4.1, 2.5, 5.6, 1.2])
print('신경망의 예측값 :', a)
print('소프트맥스 함수의 출력 :', softmax(a))
```

```
신경망의 예측값 : [0.5 4.1 2.5 5.6 1.2]
소프트맥스 함수의 출력 : [0.00473882 0.17343248 0.03501541 0.77727047
0.00954281]
```

부드러운 최대값: 소프트맥스 함수

- 이 코드의 실행 결과를 살펴보면 신경망의 예측값 0.5는 소프트맥스 함수에 의해 약 0.0047로 매칭되며, 가장 큰 값인 5.6은 약 0.7772의 값으로 매칭되는 것을 볼 수 있다.
  - 넘파이의 **set_printoptions()** 함수를 사용하면 정밀도 값을 제어하는 것도 가능하다.
  - 이 함수의 키워드 인자인 **precision**은 소수점 아래 정밀도를 제어하며, **suppress** 인자는 $1.1234e-01$과 같은 형식의 출력을 0.11234와 같이 나타내는 일을 한다.

```
np.set_printoptions(precision=4, suppress=True)
print('소프트맥스 함수의 출력 :', softmax(a))
```

```
소프트맥스 함수의 출력 : [0.0047 0.1734 0.035  0.7773 0.0095]
```

부드러운 최대값: 소프트맥스 함수

- 소프트맥스 함수의 최대값은 다음과 같이 **np.max()** 함수로 얻을 수 있으며, **np.argmax()** 함수로 이 최대값의 인덱스 3도 얻어낼 수 있다

```
print('소프트맥스 함수의 최대값 :', np.max(softmax(a)).round(4))
```

```
소프트맥스 함수의 최대값 : 0.7773
```

```
print('소프트맥스 함수 출력의 argmax :', np.argmax(softmax(a)))
```

```
소프트맥스 함수 출력의 argmax : 3
```

부드러운 최대값: 소프트맥스 함수

- 소프트맥스 함수는 지수함수이므로 입력값 a가 다음과 같이 **두 배로 커질 경우** 가장 큰 값의 확률 값이 크게 증가하는 것을 볼 수 있다.
  - 이상의 내용을 통해 **소프트맥스 함수는 최대값을 더욱 활성화 하고 작은 값을 억제** 하는 효과를 만들어내는 것을 알 수 있다.

```
# 소프트맥스 함수의 입력값을 두 배로 증가시켜 보자
a = np.array([0.5, 4.1, 2.5, 5.6, 1.2]) * 2
print('신경망의 예측값 :', a)
print('소프트맥스 함수의 출력 :', softmax(a))
print('소프트맥스 함수의 최대값 :', np.max(softmax(a)))
```

```
신경망의 예측값 : [ 1.   8.2  5.  11.2  2.4]
소프트맥스 함수의 출력 : [0.     0.0473 0.0019 0.9506 0.0001]
소프트맥스 함수의 최대값 : 0.9505663642857384
```

부드러운 최대값: 소프트맥스 함수

- 이상에서 다룬 내용을 바탕으로 분류 문제에 사용되는 소프트맥스 함수의 핵심적인 장점을 세 가지로 설명할 수 있다.

1. **확률적 해석 가능성:** 소프트맥스 함수는 각 클래스에 대한 딥러닝 모델의 예측값을 확률 점수로 제공한다. 이 점수들은 모두 0과 1 사이의 값이며, 전체 합계는 항상 1이 된다. 이는 문제의 정답이 여러 개의 범주 중 하나일 때, 모델이 그 각각의 예측에 대해 얼마나 '신뢰'하는지 정량화하는 데 유용하다.
2. **다중 클래스 분류에 유용함:** 소프트맥스 함수는 다중 클래스 분류 문제의 출력으로 이상적이다. 즉, 입력 데이터가 여러 개의 범주 중 하나에 속해야 하는 경우에 사용될 수 있다. 이 함수는 각 클래스에 대한 점수를 제공하고, 가장 높은 점수를 가진 클래스를 선택하여 분류 문제를 해결하는 데 도움이 된다.
3. **최대값을 활성화하고 최소값을 억제함:** 소프트맥스 함수는 e를 밑으로 하는 지수를 사용하기 때문에 신경망의 예측값 중에서 최대값은 더욱 크게 만들고, 최소값은 더욱 작게 만드는 효과가 있다.

부드러운 최대값: 소프트맥스 함수

- 다음 도전 문제를 풀어보자

도전 문제 7.4: 소프트맥스 함수 사용하기

상 중 하



1. 신경망의 출력층이 3개의 출력을 내보낸다고 가정하자. 이 출력층의 값이 각각 0.4, 4.5, 1.9일 경우, 소프트맥스 함수를 사용하여 이 출력값을 0에서 1 사이의 확률값으로 나타내 보자. 그리고 소프트맥스 함수의 최대값, 이 최대값의 인덱스를 argmax를 이용하여 다음과 같이 출력해 보자.

```
신경망의 예측값 : [0.4 4.5 1.9]
소프트맥스 함수의 출력 : [0.0152 0.9167 0.0681]
소프트맥스 함수의 최대값 : 0.9167
소프트맥스 함수 출력의 argmax : 1
```

2. 신경망의 출력층이 6개의 출력을 내보낸다고 가정하자. 이 출력층의 값이 각각 4.9, 2.4, 12.1, 2.5, 1.9, 0.2일 경우, 소프트맥스 함수를 사용하여 이 출력값을 0에서 1 사이의 확률값으로 나타내 보자. 그리고 소프트맥스 함수의 최대값, 이 최대값의 인덱스를 argmax를 이용하여 다음과 같이 출력해 보자.

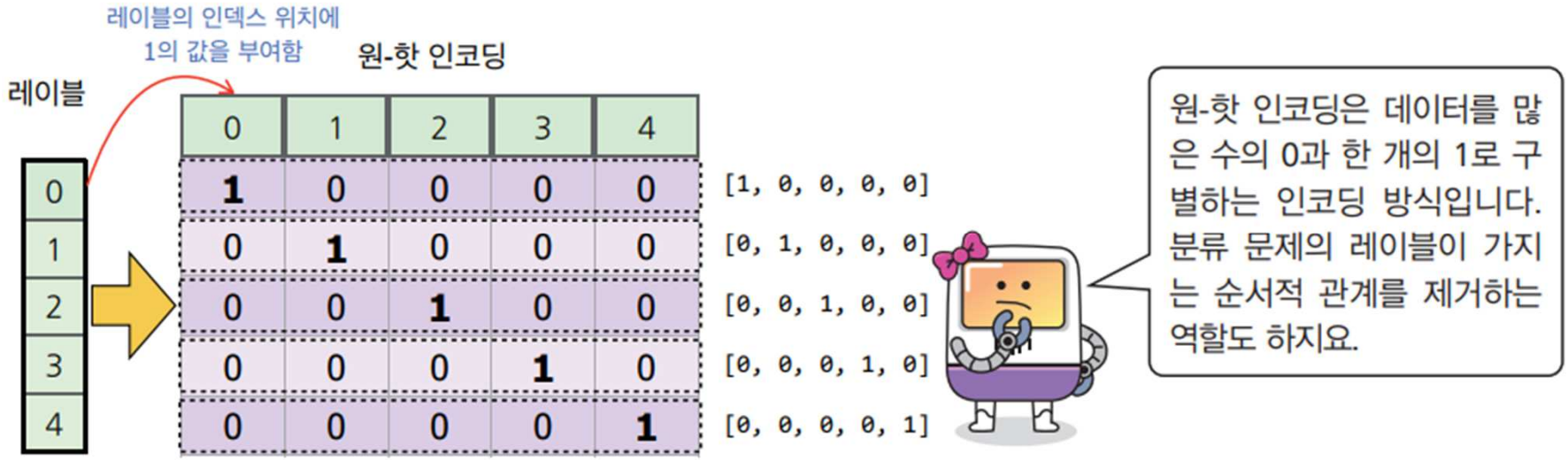
```
신경망의 예측값 : [ 4.9  2.4 12.1  2.5  1.9  0.2]
소프트맥스 함수의 출력 : [0.0007 0.0001 0.9991 0.0001 0.     0.    ]
소프트맥스 함수의 최대값 : 0.9991
소프트맥스 함수 출력의 argmax : 2
```

원-핫 인코딩과 범주형 데이터의 평균 제곱오차

- 다음으로 소프트맥스 함수의 출력값과 정답과의 차이를 구하는 방법을 살펴보자.
- 이를 위하여 우선 **원-핫 인코딩**one-hot encoding 기법의 필요성에 대하여 알아보자. 이전 절에서 살펴본 데이터와 동일한 자료와 소프트맥스 함수를 통한 출력은 $[0.0047, 0.1734, 0.0350, \mathbf{0.7772}, 0.0095]$와 같은 벡터값인 반면, 이 데이터의 정답은 $\mathbf{3}$과 같은 수치값으로 레이블링되어 있다.
- 이 모델의 출력은 $0, 1, 2, 3, 4$ 중 하나인데 인간의 눈으로 보게 되면 이 출력의 0은 1보다 더 작은 값으로 보게 된다.
  - 이것을 **순서적 관계** 또는 **서수 관계**라고 함
  - 이 순서적 관계는 모델의 범주 분류에 무관하며 이러한 **순서적 관계를 제거하면 잘못된 관계를 만드는 오류에서 벗어날 수 있다.**

원-핫 인코딩과 범주형 데이터의 평균 제곱오차

- 원-핫 인코딩은 위에서 다룬 0, 1, 2, 3, 4 또는 '빨강', '파랑', '녹색' 등과 같은 카테고리 값을 이진수의 벡터로 사상시키는 기법이다.
- 이 방법은 그림과 같이 0이라는 레이블을 나타내기 위해서 [**1**, 0, 0, 0, 0]이라는 표기를, 1이라는 레이블을 나타내기 위해서 [0, **1**, 0, 0, 0]이라는 표기를, 2라는 레이블을 나타내기 위해서 [0, 0, **1**, 0, 0]이라는 표기를 각각 사용한다.



원-핫 인코딩과 범주형 데이터의 평균 제곱오차

- 이 방법은 소프트맥스 함수를 통과한 벡터 출력값이 스칼라 형식의 정답과 얼마만큼의 차이가 나는지 정량화하는 데에도 유용하게 사용될 수 있으며, 언어 모델에서도 단어를 **원-핫 인코딩**을 통해서 나타내는 데 사용할 수 있다.
- 원-핫 인코딩은 다음 두 단계의 과정을 통해서 간단하게 구현할 수 있다.

1. 각 레이블에 고유의 인덱스를 부여한다(예: 0, 1, 2 등).
2. 표현하고자 하는 레이블의 인덱스 위치에 1을 부여하고, 나머지 인덱스의 값으로 0을 부여한다.

원-핫 인코딩과 범주형 데이터의 평균 제곱오차

- 우리가 살펴본 **keras**의 **utils** 모듈에는 다음과 같은 **to_categorical()** 함수가 제공된다.
  - [0, 1, 2, 3, 4]의 데이터가 어떻게 인코딩되는지 살펴보자.
  - 살펴본 코드의 결과와 같이 0에서 4까지의 수치 데이터가 들어 있는 **data**는 **to_categorical()** 함수를 통해서 원-핫 인코딩된 데이터로 변환될 수 있다.
  - 즉, 레이블 **3**은 원-핫 인코딩되어 [0, 0, 0, 1, 0]의 형태로 변환되는 것을 볼 수 있다.

```
import numpy as np
from keras.utils import to_categorical

data = np.array([0, 1, 2, 3, 4])  # 수치 데이터 0에서 4까지의 값
print('인코딩할 원본 데이터', data)
encoded = to_categorical(data)    # 원-핫 인코딩된 범주형 데이터 생성
print('원-핫 인코딩된 데이터 :')
print(encoded)
```

```
인코딩할 원본 데이터 [0 1 2 3 4]
원-핫 인코딩된 데이터 :
[[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]]
```

원-핫 인코딩과 범주형 데이터의 평균 제곱오차

- 만일 원본 데이터가 수치형 데이터가 아닌 **'red', 'green', 'blue'**와 같은 문자열 범주 데이터일 경우에는 어떻게 해야 할까?
  - 파이썬의 딕셔너리 자료형으로 변환한 후 이 자료형의 키 값을 수치 데이터로 사용하여 위에서 사용한 함수를 사용하면 된다.
- 아래와 같이 **color_data**라는 리스트 자료형에 **['red', 'green', 'blue']**가 있을 경우, 이를 리스트 축약 코드를 사용하여 딕셔너리로 만들도록 하자.

```
color_data = ['red', 'green', 'blue']
# 리스트 축약 코드를 사용하여 딕셔너리 자료형으로 만들자
color_dic = {v: k for v, k in enumerate(color_data)}
print('color_dic :', color_dic)
```

```
color_dic : {0: 'red', 1: 'green', 2: 'blue'}
```

원-핫 인코딩과 범주형 데이터의 평균 제곱오차

- 다음으로 이 딕셔너리 **color_dic**의 키를 추출하여 이전에 설명한 **to_category()** 함수를 통해서 원-핫 인코딩된 데이터를 만들고 이를 출력해 보도록 하자.

```
import numpy as np
from keras.utils import to_categorical

data = np.array(list(color_dic.keys()))  # 키값 0, 1, 2를 항목으로 하는 data
print('인코딩할 원본 데이터 :', data)
encoded = to_categorical(data)           # 원-핫 인코딩된 범주형 데이터 생성
print('원-핫 인코딩된 데이터 :')
print(encoded)
```

```
인코딩할 원본 데이터 : [0 1 2]
원-핫 인코딩된 데이터 :
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

```
for i in range(len(color_data)):
    print('{:5s} : {}'.format(color_data[i], encoded[i]))
```

```
red   : [1. 0. 0.]
green : [0. 1. 0.]
blue  : [0. 0. 1.]
```

원-핫 인코딩과 범주형 데이터의 평균 제곱오차

- 이제 다음과 같이 정답값 3이 원-핫 인코딩되어 있다고 가정하고 이 정답값과 소프트맥스 함수의 출력값을 바탕으로 신경망의 오차를 구해 보자.
- 여기서는 예측값 $\hat{y}$과의 차이를 **평균 제곱오차** 방식의 측정함수를 사용하여 구현할 수 있을 것이다. 아래의 코드에서 **target**은 정답값, **y_hat**은 예측값 $\hat{y}$을 각각 의미한다.

```
import numpy as np

# 정답 3이 인코딩된 target과 신경망의 예측값 y_hat이 있다고 가정하자
target = np.array([0, 0, 0, 1, 0])
y_hat = np.array([0.005, 0.173, 0.035, 0.777, 0.01])

def mse(y, t):   # 평균 제곱오차
    return ((y - t)**2).mean()

print('y_hat과 target과의 오차 :', mse(y_hat, target))
```

```
y_hat과 target과의 오차 : 0.016201599999999997
```

원-핫 인코딩과 범주형 데이터의 평균 제곱오차

- 평균 제곱오차 함수는 범주형 데이터에 대하여, 다음과 같이 정답 레이블과 예측값의 차이를 구한 다음, 이들 각각에 대하여 제곱의 합을 구하고 그 결과를 전체 샘플의 수 m으로 나누어 계산한다.
- 따라서 y_i 데이터에 대한 예측값을 $\hat{y}_i$ 라고 할 때 E_{mse}는 다음과 같은 식으로 나타낼 수 있다.

$$E_{mse} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$$

원-핫 인코딩과 범주형 데이터의 평균 제곱오차

- 이 식은 신경망의 예측값($\hat{y}_i$)과 정답값(y_i)의 차이를 구하여 제곱한 수들의 합을 전체 데이터의 수로 나눈 값이다.
  - 이전 코드는 정답 3에 대한 확률값이 0.777로 전체 값 중에서 가장 크기 때문에 0.0162의 비교적 작은 오차값을 출력하였다.
  - 만일 신경망의 예측값이 [0.1, 0.3, 0.5, **0.02**, 0.08]과 같이 정답에서 벗어날 경우, 평균 제곱오차의 값은 0.26336처럼 이전 값에 비해 큰 값이 된다.

```
# 정답에서 많이 벗어난 other_y_hat 추정치
other_y_hat = np.array([0.1, 0.3, 0.5, 0.02, 0.08])
# other_y_hat 추정치와 정답과의 오차를 알아보자
print('other_y_hat과 target과의 오차 :', mse( other_y_hat, target ))
```

```
other_y_hat과 target과의 오차 : 0.26336
```

- **정답에서 많이 벗어날수록 큰 오차값을 출력**해야 신경망의 학습이 빠르게 이루어진다는 측면에서 이러한 오차 측정 방식은 매우 합리적이다.

원-핫 인코딩과 범주형 데이터의 평균 제곱오차

- 원-핫 인코딩의 핵심적인 장점은 다음과 같이 요약할 수 있다.

범주형 데이터를 수치화: 원-핫 인코딩을 사용하면 범주형 데이터를 컴퓨터가 이해할 수 있는 숫자 형태로 변환할 수 있다. 예를 들어, '빨강', '파랑', '녹색' 등의 색상 범주를 [1,0,0], [0,1,0], [0,0,1]과 같은 벡터로 표현할 수 있다. 이는 컴퓨터가 데이터를 처리하고 학습하는 데 필수적인 과정이다.

순서에 따른 관계를 제거: 범주형 변수를 단순히 숫자로 변환하면 잘못된 관계나 순서가 데이터에 부여될 수 있다. 예를 들어, '빨강'=1, '파랑'=2, '녹색'=3과 같이 인코딩하면, 녹색이 파랑보다 '큰' 것으로 해석될 수 있다. 원-핫 인코딩은 이러한 순서에 따른 관계를 제거하고 각 범주를 독립적인 이진 특징으로 표현함으로써 이 문제를 해결한다.

원-핫 인코딩과 범주형 데이터의 평균 제곱오차

- 다음 도전 문제를 풀어보자

도전 문제 7.5: 범주형 데이터를 원-핫 인코딩하기

상 중 하



머신러닝과 딥러닝에서 사용되는 데이터 중에서 Iris 데이터 셋은 주어진 붓꽃의 꽃잎과 꽃받침의 너비, 높이를 이용하여 Versicolor, Setosa, Virginica의 세 종을 분류하는 것을 목적으로 하고 있다. 이 세 종의 레이블을 넘파이와 사이킷런 모듈을 사용하여 원-핫 인코딩하고 그 결과를 다음과 같이 출력하여라.

```
Versicolor : [1. 0. 0.]
Setosa     : [0. 1. 0.]
Virginica  : [0. 0. 1.]
```

원-핫 인코딩 데이터의 오차를 구하는 교차 엔트로피 오차

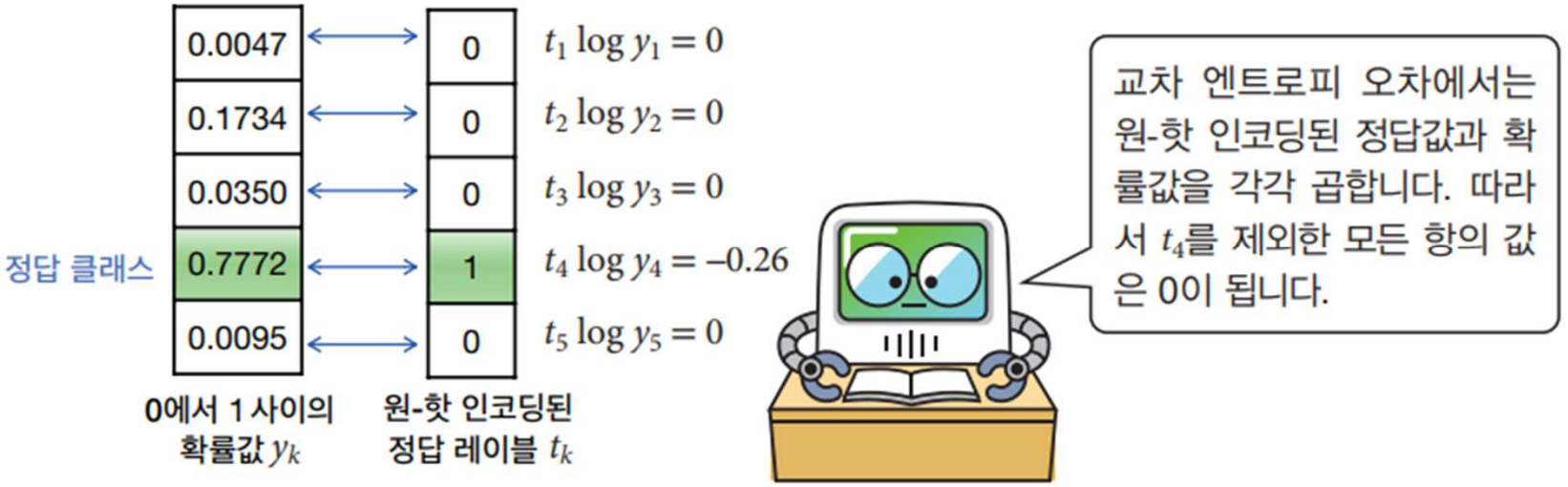
- 소프트맥스 함수나 평균 제곱오차 함수보다도 더 나은 오차 평가 방법은 없을까?
  - **교차 엔트로피 오차**cross entropy error: cee 는 분류 모델을 평가할 때 평균 제곱오차 방식보다 더 유용하게 사용될 수 있다.
  - 교차 엔트로피 오차 평가 함수를 사용하기 위해서는 **분류 모델의 정답값이 원-핫 인코딩으로 구성**되어 있어야만 한다.
- 교차 엔트로피 오차는 앞으로E_{cee}로 표기할 것이다.
  - 다음과 같이 로그의 밑이 e인 자연로그를 예측값 y_k에 씌워서 실제 값 t_k와 곱한 후 전체 값을 합한 후 음수로 변환한다. (밑을 생략한 $\log(x)$가 자연로그 $\ln(x)$를 의미)

$$E_{cee} = -\sum_k t_k \log y_k$$

- 이 수식을 살펴보면 정답값이 원-핫 인코딩 방식일 경우에는 정답 위치의 확률 값을 제외한 나머지는 무조건 0이 나오므로 이 값을 합하게 되더라도 결과에 영향을 주지 않으며, 오로지 **정답값에 대한 예측값만이 결과에 영향을 주는 식**이 된다.

원-핫 인코딩 데이터의 오차를 구하는 교차 엔트로피 오차

- 예를 들어, 그림과 같이 정답 클래스가 네 번째이고 소프트맥스 함수가 네 번째 클래스의 확률 값을 0.77로 예측했다면 교차 엔트로피 오차는 $-\log(0.77) = 0.26$이 된다.
  - t_4항목을 제외한 나머지 모든 항에 대해서 0이 곱해지기 때문임.
  - 그러나 만일 정답값이 세 번째일 경우에는 $-\log(0.035) = 3.352$의 비교적 큰 오차값을 출력할 것이다. 즉, 예측값이 정답에 가까울 경우 E_{cee}는 작은 오차값을 출력하고, 예측값이 정답값에서 멀어질수록 큰 오차값을 출력하는 특징이 있다.



원-핫 인코딩 데이터의 오차를 구하는 교차 엔트로피 오차

- 예시를 통해 **교차 엔트로피 오차가 신경망의 오차를 측정하기에 더욱더 적절한 이유**를 살펴보자.
  - 우선 신경망에서 출력한 두 개의 추정값이 존재한다고 가정하자. 이해를 돕기 위해 3개의 클래스로 단순화하였다.
  - 첫 번째 추정값 $\hat{Y}_1$이 $[0.4, 0.05, 0.55]$이고, 두 번째 추정값 $\hat{Y}_2$가 $[0.9, 0.09, 0.01]$이라고 가정하자.
  - 이번에는 정답이 2로 $[0, 0, 1]$과 같이 인코딩 되어 있을 경우를 가정하자.
  - 이 경우 정답값 2에 대한 추정값이 더 크게 나오도록 학습이 진행되는 것이 바람직하다.
  - 이제 **cee()** 함수를 구현하고 두 추정값의 오차를 각각 비교해 보도록 하자.(다음 페이지)

```
# 교차 엔트로피 오차 함수를 에러 함수로 사용해 보도록 하자
def cee(y, t):
    delta = 1e-7                        # 0이 되지 않도록 작은 마진 값을 준다
    return -np.sum(t * np.log(y + delta))

t = np.array([0, 0, 1])                 # 세 번째 레이블이 정답값
y_hat1 = np.array([0.4, 0.05, 0.55])
y_hat2 = np.array([0.9, 0.09, 0.01])

# y_hat 추정치와 정답과의 CEE 오차를 알아보자
print('y_hat1과의 cee : {:.2f}'.format(cee(y_hat1, t)))
print('y_hat2과의 cee : {:.2f}'.format(cee(y_hat2, t)))
```

원-핫 인코딩 데이터의 오차를 구하는 교차 엔트로피 오차

- $\hat{Y}_1$이 $\hat{Y}_2$에 비해서 더 좋은 추정치이다.
- 평균 제곱오차와 교차 엔트로피 오차 모두 y_hat2에 비해 y_hat1의 오차값을 더 낮게 출력하는 것을 볼 수 있다.

```python
# 교차 엔트로피 오차 함수를 에러 함수로 사용해 보도록 하자
def cee(y, t):
    delta = 1e-7                        # 0이 되지 않도록 작은 마진 값을 준다
    return -np.sum(t * np.log(y + delta))

t = np.array([0, 0, 1])                 # 세 번째 레이블이 정답값
y_hat1 = np.array([0.4, 0.05, 0.55])
y_hat2 = np.array([0.9, 0.09, 0.01])

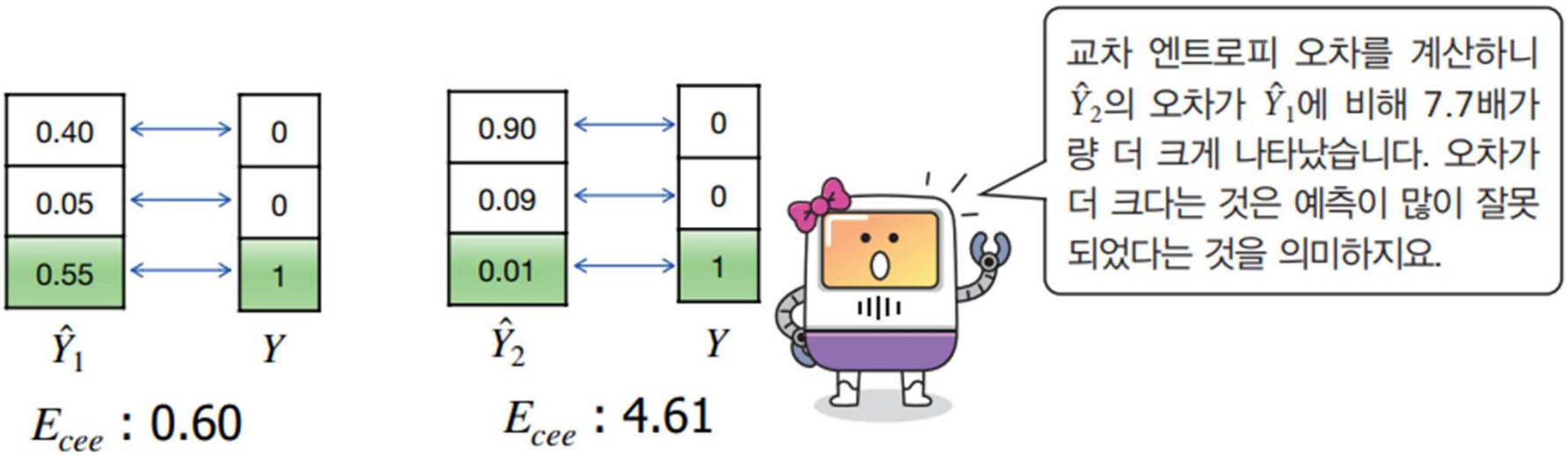
# y_hat 추정치와 정답과의 CEE 오차를 알아보자
print('y_hat1과의 cee : {:.2f}'.format(cee(y_hat1, t)))
print('y_hat2과의 cee : {:.2f}'.format(cee(y_hat2, t)))
print('두 값의 비 : {:.2f}'.format(cee(y_hat2, t)/cee(y_hat1, t)))

# y_hat 추정치와 정답과의 MSE 오차를 알아보자
# mse() 함수는 이전에 구현한 것과 동일하다
print('y_hat1과의 mse : {:.2f}'.format(mse(y_hat1, t)))
print('y_hat2과의 mse : {:.2f}'.format(mse(y_hat2, t)))
print('두 값의 비 : {:.2f}'.format(mse(y_hat2, t)/mse(y_hat1, t)))
```

```
y_hat1과의 cee : 0.60
y_hat2과의 cee : 4.61
두 값의 비 : 7.70
y_hat1과의 mse : 0.12
y_hat2과의 mse : 0.60
두 값의 비 : 4.93
```

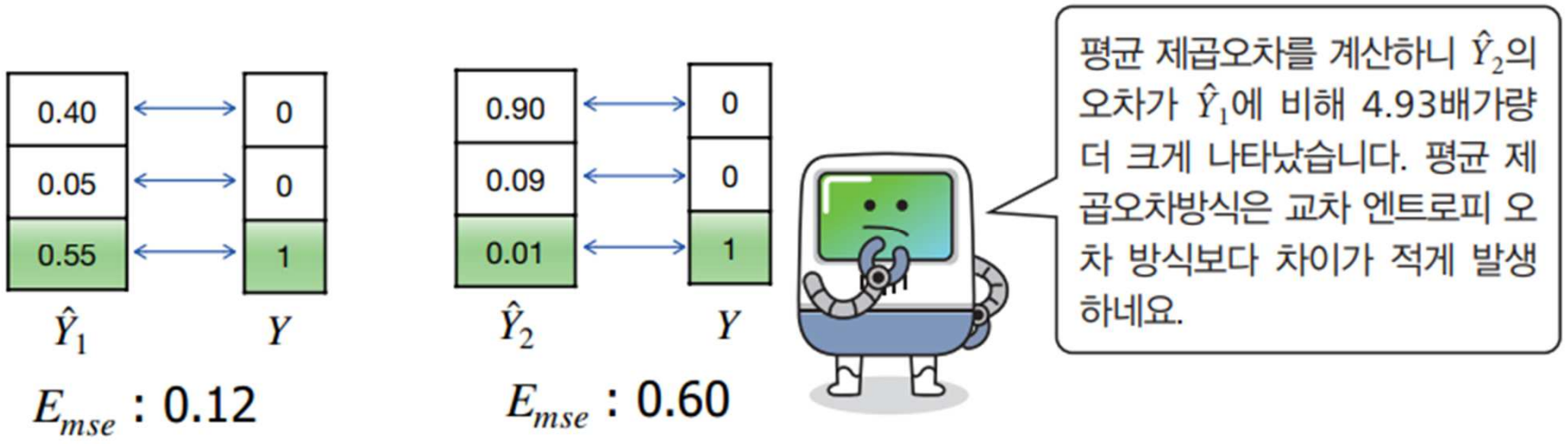
원-핫 인코딩 데이터의 오차를 구하는 교차 엔트로피 오차

- 교차 엔트로피 오차의 경우 y_hat1의 오차가 0.60, y_hat2의 오차가 4.61로 나타났다.
  - 이 값의 비율은 1: 7.7로, y_hat2의 예측값이 y_hat1에 비해 더 부정확하다는 의미이다.
- 그 이유는 정답 레이블에 대하여 y_hat1은 0.55로 예측했고 y_hat2는 0.01로 예측했기 때문이다.



원-핫 인코딩 데이터의 오차를 구하는 교차 엔트로피 오차

- 이제 평균 제곱오차에서는 어떻게 예측했는지 살펴보자. 이 방식은 y_hat에 대하여 0.12, y_hat2 에 대하여 0.61로 오차를 알려준다.
- 비록 y_hat1이 더 오차가 작기는 하지만 이 두 값의 비율은 4.93배로, 교차 엔트로피 오차에서 구한 오차값의 차이보다는 더 적다.



원-핫 인코딩 데이터의 오차를 구하는 교차 엔트로피 오차

- 즉, 평균 제곱오차는 첫 번째 추정치의 오차를 두 번째 추정치보다 4.93배 더 낮게 추정하는데 비해, 교차 엔트로피 오차는 7.7배나 더 낮게 추정하고 있다.
  - 신경망은 학습의 오류를 줄이는 과정이기 때문에 **벌점에 해당하는 오차값이 더 큰 교차 엔트로피 오차를 신경망의 오차로 사용하는 것이 학습의 속도를 더 빠르게** 할 수 있다.

- 교차 엔트로피 오차 방식은 분류할 레이블이 매우 많을 경우에도 효과적이다.
  - 이 방식은 1의 값을 가지는 정답 레이블과의 오차만을 계산하므로, **정답이 아닌 레이블(tk = 0인 경우)과의 오차를 구할 필요가 없어 평균 제곱오차보다 계산량이 줄어든다(최신 컴파일러는 0*n의 계산값을 자동으로 0으로 계산하는 성능 최적화 기법을 사용하기 때문이다).**

최적화 기법

- 위에서 살펴본 신경망 모델에서 compile() 메소드의 키워드 인자 중 하나인 optimizer에 대해서 살펴보자.
- **최적화 기법**이란 외래어 표기를 그대로 사용하여 **옵티마이저**라고도 하는데 **알고리즘이나 모델이 최적의 성능을 내기 위한 매개변수를 찾는 데 사용되는 도구 또는 방법**을 말한다. 다시 말해, 옵티마이저는 모델의 손실 함수를 최소화하거나, 정확도와 같은 성능 지표를 최대화하는 매개변수 값을 찾는 역할을 한다.
- 딥러닝에서는 대표적으로 **경사하강법**이라는 방법을 사용하는데, 이는 매개변수를 손실 함수의 기울기에 비례하여 조정함으로써 손실을 줄이는 방법이다. 경사 하강법에도 다양한 변형들이 존재하는데, 이들은 각각의 구조상에서 성능을 향상시키는 방법을 제공한다.
- 가장 널리 사용되는 옵티마이저에는 **확률적 경사하강법** Stochastic Gradient Descent: SGD, **모멘텀, RMSProp, Adam** 등이 있다.

최적화 기법

- 이러한 옵티마이저의 선택은 모델의 성능에 매우 중요한 영향을 미치므로, 옵티마이저의 종류와 특징을 이해하고 적절하게 선택하는 것이 중요하다.
- 딥러닝의 최적화를 위해서 다양한 방법이 필요한 이유

  - **문제의 복잡도:** 딥러닝 모델은 파라미터가 수백 개 정도인 단순한 모델부터 수백억 개 이상의 방대한 모델이 존재한다. 이 파라미터들의 조합이 만드는 손실 함수의 표면은 매우 복잡하다. 경우에 따라서 이 표면에는 여러 지역 최소값, 안장점, 평평한 지역 등이 있을 수 있다. 따라서 단순한 경사하강법만으로는 이러한 복잡한 표면에서 최적의 해를 찾는 데 어려움이 있을 수 있다. 그래서 다양한 종류의 옵티마이저들이 등장했으며, 이들은 각각 다른 전략을 사용하여 이러한 문제를 해결한다.
  - **학습 속도와 안정성의 차이:** 지금까지 연구되어 온 옵티마이저마다 각자의 장단점이 있다. 예를 들어, 일부 옵티마이저는 학습 속도가 빠르지만, 학습 과정이 불안정하거나 최적의 해를 찾지 못할 수 있다. 반면에, 다른 옵티마이저는 학습 속도가 느리지만 안정적인 결과를 보장하거나, 더 좋은 해를 찾을 가능성이 있을 수 있다. 따라서 특정 문제에 대해 가장 적합한 옵티마이저를 선택하려면 여러 가지 옵티마이저 중에서 선택할 수 있어야 한다.

최적화 기법

- 결국, 딥러닝 모델을 학습시킬 때 이 모델에 맞는 적절한 옵티마이저를 선택하게 되면 더 빠르고 안정적이며, 더 나은 성능의 결과를 얻는데 도움이 된다.
- 딥러닝 모델의 학습을 위해서 사용하는 용어 정리

  - **에폭:** 주어진 훈련 데이터 셋에 대해서 딥러닝 알고리즘이 실행되는 횟수를 말한다. 딥러닝 모델은 주어진 훈련 데이터를 입력으로 받아서 가장 좋은 성능의 파라미터를 만들어 내는 일을 한다.
  - **샘플:** 샘플이란 데이터의 기본 단위이다. 예를 들어, 이미지 분류 작업을 수행하기 위해서는 각각의 이미지가 하나의 샘플이 된다. 각 이미지는 특정 레이블(예: 강아지, 고양이, 자동차 등)과 연결되어 있다.
  - **배치:** 모델 파라미터를 업데이트하기 위해 사용할 샘플의 수를 나타낸다. 보통 여러 개의 샘플을 딥러닝 알고리즘을 학습시키는 데 사용하며 이 배치의 크기가 크면 클수록 학습에 소요되는 시간과 비용은 증가한다. 따라서 딥러닝 모델을 다루는 엔지니어는 최선의 배치 크기를 찾도록 노력해야 한다.
  - **학습률:** 모델을 학습시키는 하이퍼파라미터의 하나로, 모델 가중치가 어느 정도의 척도로 갱신되어야 하는지 지정하는 일을 한다. 이 학습률이 너무 작으면 모델은 느리게 학습되며, 이 학습률이 너무 클 경우 모델이 제대로 학습되지 않고 발산되는 경우가 발생한다.
  - **비용 함수:** 손실 함수라고도 하며 예측 값과 실제 값의 차이인 비용을 계산하는 함수이다.
  - **가중치:** 인공신경망의 퍼셉트론끼리 신호를 제어하는 모델에서 학습 가능한 파라미터(매개변수)를 말한다.

최적화 기법

- 널리 사용되는 옵티마이저의 특징과 장단점을 간략히 설명하면 다음과 같다.

1. 경사하강법(Gradient Descent)
   - 설명: 전체 데이터 셋에 대하여 오차를 구하고 가중치를 갱신한다.
   - 장점: 구현이 쉽다.
   - 단점: 매번 전체 데이터 셋의 오차를 구해야 하므로 수렴 속도가 매우 느리다.
2. 확률적 경사하강법(Stochastic Gradient Descent)
   - 설명: 전체 데이터 셋에서 샘플을 무작위로 선택하여 가중치를 갱신한다.
   - 장점: 구현이 쉽다.
   - 단점: 수렴 속도가 느리고, 때로는 최적점에 수렴하지 못할 수도 있다.
3. 모멘텀(Momentum)
   - 설명: 이전 단계의 그래디언트 방향을 기억하고 있다가 이를 현재 단계의 그래디언트에 추가하여 가중치를 갱신한다. 이로써 **관성** 효과를 얻을 수 있다.
   - 장점: 지역 최적점에서 벗어나는 데 도움을 줄 수 있다.
   - 단점: 추가적인 하이퍼파라미터 설정이 필요하다.

최적화 기법

- 널리 사용되는 옵티마이저의 특징과 장단점을 간략히 설명하면 다음과 같다.

4. Adagrad
   - 설명: 각 가중치에 대해 학습률을 조정하여 갱신한다.
   - 장점: 특정 가중치에 따라 학습률을 다르게 조정할 수 있다.
   - 단점: 학습 중 학습률이 너무 빨리 감소할 수 있다.

5. RMSprop
   - 설명: Adagrad의 문제를 해결하기 위해 도입된 방법으로 가장 최근의 그래디언트만 사용하여 학습률을 조정한다.
   - 장점: 다양한 문제에 대해 잘 작동한다.
   - 단점: 학습률이 수행할 작업에 따라 유동적이므로 특정 문제에 대하여 최적화되도록 그 값을 조절할 필요가 있다.

6. Adam
   - 설명: 모멘텀과 RMSprop의 아이디어를 결합한 옵티마이저이다.
   - 장점: 다양한 문제와 데이터에 대해 일반적으로 잘 작동한다.
   - 단점: 때로는 불안정한 학습 동작을 보일 수 있다.

최적화 기법

- 이러한 옵티마이저들 외에도 여러 다양한 옵티마이저들이 존재함.
  - 딥러닝 프로젝트에서는 주로 문제와 데이터의 특성에 맞는 옵티마이저를 선택하여 사용하게 된다.
- 옵티마이저를 지정하는 것은 다음 두 가지 방식 중 하나를 선택할 수 있다.

```
# 옵티마이저 - 확률적 경사하강법을 사용
model.compile(optimizer='SGD', loss='mean_squared_error',...)
```

```
# 옵티마이저 - 확률적 경사하강법
sgd = tf.keras.optimizers.SGD(learning_rate=0.01)

# 비용/손실 함수를 mean_squared_error 방식을 사용한다, mse라고 써도 인식한다
model.compile(optimizer=sgd, loss='mean_squared_error',...)
```

- 위의 설명에 나오는 **지역 최적점**이란 특정 최적화 함수 입장에서 가장 낮은 오차값에 해당하지만, 전체 함수에서 보면 가장 낮은 오차값이 아닌 곳을 의미한다.
- 학습 단계에서 오차 함수가 지역 최적점에 갇히게 되면, 실제로 가능한 최고의 성능에 도달하지 못할 수 있다.

요약

01 **텐서플로**는 머신러닝 모델을 설계, 훈련, 배포하는데 사용되는데, 다양한 API와 데이터까지 제공한다.

02 머신러닝과 딥러닝에서 사용되는 데이터 셋인 MNIST는 Modified National Institute of Standards and Technology의 약어로, 미국 인구 조사국에서 수집한 손글씨 숫자 데이터 집합을 머신러닝에 적합하도록 수정한 이미지 집합이다.

03 **딥러닝**은 인공신경망의 **층**을 **계층적으로 깊이 쌓아서 만든 신경망 모델을 통해서 학습을 하는 알고리즘**을 말한다.

04 텐서플로에서 사용하는 신경망층의 이름인 Dense는 학습을 위한 연결을 **밀집된** 구조 혹은 **완전 연결**층으로 한다는 의미이다. Dense 네트워크의 입력은 앞 층에서 주어지기 때문에 몇 개의 출력으로 연결할지를 정하는 매개변수가 존재한다.

05 모델을 점점 더 좋은 상태로 만드는 것을 **최적화**라고 하며, 현재 모델의 정답과 실제 정답의 차이가 오차이고 이 오차를 측정하는 것이 **손실 함수**이다.

06 학습 데이터 모음을 가지고 한 번 훈련을 실시하는 것을 **에폭**이라고 부르는데, 이것을 반복함으로써 모델은 더 좋은 상태로 바뀌게 된다.

07 **가중치 값을 가진 벡터를 0에서 1 사이의 클래스별 확률값으로 변환**하기 위해서 사용하는 함수가 바로 **소프트맥스** 함수이다.

08 **원-핫 인코딩**은 각 레이블에 고유의 인덱스를 부여한 뒤에 표현하고자 하는 레이블의 인덱스 위치에 1을 부여하고, 나머지 인덱스의 값으로 0을 부여하는 단계를 통해 간단하게 구현이 가능하다.

09 **교차 엔트로피 오차는 분류 모델**을 평가할 때 **평균 제곱오차 방식**보다 더 유용하게 사용될 수 있다. 이 오차 평가 함수를 사용하기 위해서는 **분류 모델의 정답값이 원-핫 인코딩으로 구성**되어 있어야만 한다.